

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

JOÃO VICTOR BRIGANTI DE OLIVEIRA

**EXTENSÃO DO OPENMP PARA SUPORTE DE TÉCNICAS DE
APROXIMAÇÃO**

CAMPO MOURÃO

2026

JOÃO VICTOR BRIGANTI DE OLIVEIRA

**EXTENSÃO DO OPENMP PARA SUPORTE DE TÉCNICAS DE
APROXIMAÇÃO**

OpenMP Extension for Approximate Computing Techniques

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção do
título de Bacharel em Ciência da Computação
do Curso de Bacharelado em Ciência da
Computação da Universidade Tecnológica
Federal do Paraná.

Orientador(a): Prof. Dr. João Fabrício Filho

Coorientador(a): Prof. Dr. Rogério Aparecido
Gonçalves

CAMPO MOURÃO

2026



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

JOÃO VICTOR BRIGANTI DE OLIVEIRA

**EXTENSÃO DO OPENMP PARA SUPORTE DE TÉCNICAS DE
APROXIMAÇÃO**

Trabalho de Conclusão de Curso de Graduação
apresentado como requisito para obtenção do
título de Bacharel em Ciência da Computação
do Curso de Bacharelado em Ciência da
Computação da Universidade Tecnológica
Federal do Paraná.

Data de aprovação:

João Fabrício Filho
Doutorado
Universidade Tecnológica Federal do Paraná

Rogério Aparecido Gonçalves
Doutorado
Universidade Tecnológica Federal do Paraná

Juliano Henrique Foleis
Doutorado
Universidade Tecnológica Federal do Paraná

Lucas Francisco Wanner
Doutorado
Universidade Estadual de Campinas

CAMPO MOURÃO

2026

AGRADECIMENTOS

Primeiramente, agradeço a minha família, em especial ao meu pai e minha mãe, que foram meu grande apoio ao longo de toda essa jornada, sem os seus ensinamentos e apoio nada disso seria possível.

Agradeço aos meus orientadores Prof. Dr. João Fabrício Filho e ao Prof. Dr. Rogério Aparecido Gonçalves, pelas suas orientações e contribuições valiosas, não só neste trabalho como também ao longo de toda minha Iniciação Científica.

Também agradeço a minha namorada pela sua companhia e estar ao meu lado nos momentos difíceis.

Enfim, agradeço a todos que direta ou indiretamente me ajudaram e estiveram no meu lado durante minha caminhada.

“Nada lhe posso dar que já não exista em você mesmo. Não posso abrir-lhe outro mundo de imagens, além daquele que há em sua própria alma. Nada lhe posso dar a não ser a oportunidade, o impulso, a chave. Eu o ajudarei a tornar visível o seu próprio mundo, e isso é tudo.” (Hesse, 2025).

RESUMO

Com o fim da Lei de Moore e da escala de Dennard, diferentes abordagens computacionais surgiram para suprir a crescente demanda por desempenho e eficiência energética. A Computação Aproximada busca aumentar o desempenho e reduzir o consumo energético de aplicações em troca de perdas controladas de precisão nos resultados. Já a Computação Paralela explora de forma mais eficiente os processadores *multicore* para acelerar a execução de aplicações. Nesse contexto, o OpenMP destaca-se como uma ferramenta que simplifica o desenvolvimento de aplicações paralelas por meio de anotações no código-fonte. O objetivo deste trabalho é melhorar o desempenho e a eficiência energética de aplicações por meio da implementação de técnicas de aproximação no OpenMP. As modificações propostas envolvem tanto a infraestrutura do *runtime* quanto o processo de tradução de código do OpenMP na infraestrutura do LLVM. Entre as técnicas implementadas estão a perfuração de laços, a memoização aproximada e o relaxamento de ponto flutuante. As abordagens propostas foram avaliadas em um conjunto de sete aplicações tolerantes à aproximação, permitindo analisar os impactos das aproximações no desempenho e na precisão dos resultados. Os experimentos demonstraram ganhos de desempenho de até $80\times$ em comparação com a execução sem aproximação do mesmo algoritmo. Entretanto, esses ganhos foram acompanhados por elevados níveis de erro em determinados cenários, chegando a casos em que a degradação da qualidade da saída atingiu até 100%.

Palavras-chave: paralelismo; llvm; alto desempenho; anotações de código; computação aproximada.

ABSTRACT

With the end of Moore's Law and Dennard scaling, different computational approaches have emerged to meet the growing demand for performance and energy efficiency. Approximate Computing aims to increase application performance and reduce energy consumption in exchange for controlled losses in result precision. Parallel Computing, in turn, makes more efficient use of multicore processors to accelerate application execution. In this context, `OpenMP` stands out as a tool that simplifies the development of parallel applications through source-code annotations. The goal of this work is to improve the performance and energy efficiency of applications through the implementation of approximation techniques in `OpenMP`. The proposed modifications involve both the runtime infrastructure and the code translation process of `OpenMP` within the `LLVM` infrastructure. The implemented techniques include loop perforation, approximate memoization, and floating-point relaxation. The proposed approaches were evaluated using a set of seven approximation-tolerant applications, allowing the analysis of the impact of approximations on both performance and result accuracy. The experiments demonstrated performance gains of up to $10\times$ compared to the execution of the same algorithm without approximation. However, these gains were accompanied by high error levels in certain scenarios, reaching cases in which the degradation in output quality attained up to 100%.

Keywords: parallelism; `llvm`; high performance; code annotations; approximate computing.

LISTA DE ILUSTRAÇÕES

Figura 1 – Modelo de execução do algoritmo de perfuração de laço.	16
Figura 2 – Arquitetura do OpenMP.....	20
Figura 3 – Modelo <i>fork-join</i> de execução do OpenMP.....	22
Figura 6 – Especificação do computador utilizado na avaliação experimental.....	31

LISTA DE TABELAS

Tabela 1 – Resumo e comparação entre os trabalhos relacionados.	24
Tabela 2 – Representação da correlação entre as diferentes variáveis.....	36
Tabela 3 – Resumo das aplicações utilizadas nos experimentos e seus respectivos tamanhos de entrada.	47

LISTAGEM DE CÓDIGOS FONTE

Código 1 – Estrutura de um laço canônico.	16
Código 2 – Implementação sequencial de um produto escalar.	20
Código 3 – Implementação paralelizada com <code>pthread</code> s de um produto escalar.	21
Código 4 – Implementação paralelizada com OpenMP de um produto escalar.	21
Código 5 – Sintaxe do construtor <code>approx</code>	25
Código 6 – Sintaxe da cláusula <code>perfo</code>	26
Código 7 – Exemplo de uso da cláusula <code>perfo</code>	26
Código 8 – Sintaxe da cláusula <code>fastmath</code>	27
Código 9 – Exemplo de aplicação da cláusula <code>fastmath</code>	27
Código 10 – Sintaxe da cláusula <code>memo</code>	28
Código 11 – Exemplo de uso da cláusula <code>memo</code>	30
Código 12 – Multiplicação de matrizes.	33
Código 13 – Construção da matriz de correlação.	35
Código 14 – Cálculo do coeficiente de correlação de Pearson.	35
Código 15 – Algoritmo Deriche de suavização 2D.	37
Código 16 – Método de Jacobi em duas dimensões.	40
Código 17 – Algoritmo K-means.	41
Código 18 – Função <code>find_nearest_point</code>	42
Código 19 – Cálculo do conjunto de Mandelbrot.	44
Código 20 – Geração da imagem do conjunto de Mandelbrot.	45
Código 21 – Cálculo de PI pelo método de Monte Carlo.	47

LISTA DE ABREVIATURAS E SIGLAS

API	<i>Application Programming Interface</i>
CA	Computação Aproximada
CP	Computação Paralela
GPU	<i>Graphics Processing Unit</i>
HPC	<i>High-Performance Computing</i>
IIR	<i>Infinite Impulse Response</i>
MAPE	<i>Mean Absolute Percentage Error</i>
MCR	<i>Miss-Classification Rate</i>
SIMD	<i>Single Instruction Multiple Data</i>
SSIM	<i>Structural Similarity Index Measure</i>

SUMÁRIO

1	INTRODUÇÃO	12
1.1	Objetivos	13
1.2	Contribuições	13
1.3	Organização do Trabalho	14
2	FUNDAMENTAÇÃO TEÓRICA	15
2.1	Computação Aproximada	15
2.1.1	Perforação de Laço	16
2.1.2	Memoização	16
2.1.3	Aproximação de Ponto Flutuante	17
2.2	Impacto da Aproximação no Resultado de Computações	17
2.3	Computação Paralela	19
2.4	Trabalhos Relacionados	22
3	PROPOSTA	25
3.1	Perforação de Laço	25
3.2	Aproximação de Ponto Flutuante	27
3.3	Memoização	28
4	MATERIAIS E MÉTODOS	31
4.1	Ambiente e Configuração Experimental	31
4.2	Conjunto de <i>Benchmarks</i>	32
4.2.1	2MM	33
4.2.2	Correlação	34
4.2.3	Deriche	36
4.2.4	Jacobi 2D	38
4.2.5	K-means	40
4.2.6	Mandelbrot	43
4.2.7	PI de Monte Carlo	45
4.3	Resumo do conjunto de aplicações	47
5	RESULTADOS	49
5.1	Análise por Aplicação	49
5.1.1	2MM	49
5.1.2	Correlação	51
5.1.3	Deriche	52
5.1.4	Jacobi 2D	54
5.1.5	K-Means	55
5.1.6	Mandelbrot	57
5.1.7	PI de Monte Carlo	58
5.2	Avaliação das Técnicas Empregadas	60
6	CONSIDERAÇÕES FINAIS	61
	REFERÊNCIAS	62

1 INTRODUÇÃO

Por décadas, a Lei de Moore e a escala de Dennard impulsionaram melhorias exponenciais no desempenho computacional e na eficiência energética. A Lei de Moore permitiu aumentos constantes na densidade de transistores ao longo dos anos. Já a escala de Dennard, introduzida posteriormente, garantiu por um período que esses aumentos não viessem acompanhados de maior consumo de energia. Com o avanço tecnológico, cada uma dessas tendências atingiu seus limites, tornando os ganhos de desempenho cada vez mais difíceis (Hennessy; Patterson, 2019). Mesmo com essas limitações, a crescente demanda por maior desempenho computacional e eficiência energética tem impulsionado inovações em uma ampla variedade de plataformas, desde sistemas embarcados até *datacenters*. Essa demanda é impulsionada pela crescente complexidade das cargas de trabalho modernas, como análise de dados em tempo real, aprendizado de máquina e processamento de sinais, que exigem vastos recursos computacionais (Mittal, 2016; Dalloo *et al.*, 2024).

Para atender a essas demandas sob restrições de desempenho e eficiência energética, pesquisadores têm explorado paradigmas alternativos de computação. A Computação Aproximada (CA) busca equilibrar precisão computacional e eficiência de recursos, partindo do princípio de que muitas aplicações toleram resultados com imprecisão em limites aceitáveis, definidos pela percepção do usuário ou por métricas de qualidade específicas (Dalloo *et al.*, 2024). Ao considerar que muitas aplicações não exigem resultados absolutamente precisos, a CA possibilita ganhos expressivos em desempenho, consumo de energia e utilização de *hardware* (Xu; Mytkowicz; Kim, 2016; Leon *et al.*, 2025a; Leon *et al.*, 2025b).

Além da flexibilização da precisão computacional, outra estratégia amplamente adotada para melhorar o desempenho foi a exploração do paralelismo. A Computação Paralela (CP) se mostrou alternativa viável para ganho de desempenho principalmente nas últimas décadas, nas quais houve estagnação, e aumento de frequência dos processadores devido a barreiras de potência. Para continuar evoluindo, a indústria adotou arquiteturas *multicore*, nas quais múltiplas unidades de processamento trabalham simultaneamente. Nesse contexto, a CP é empregada para acelerar aplicações dividindo seu trabalho em partes que podem ser executadas em paralelo, reduzindo o tempo total de execução. No entanto, programar para essas arquiteturas não é trivial, o desenvolvedor precisa lidar com criação e sincronização de fluxos de execução, comunicação entre tarefas e gerenciamento de memória. Ferramentas como o OpenMP surgiram justamente para abstrair essa complexidade, oferecendo um modelo de programação portátil baseado em diretivas que permitem paralelizar o código incrementalmente, sem comprometer a legibilidade, a manutenção ou a compatibilidade entre diferentes arquiteturas (Gonçalves *et al.*, 2016; Alrawais, 2021). Assim, o OpenMP consolidou-se como uma das principais ferramentas de paralelização, especialmente em ambientes de *High-Performance Computing* (HPC), nos quais simulações científicas e aplicações numéricas dependem fortemente de execução eficiente (Adhikari *et al.*, 2012).

Diante deste cenário, técnicas de CA e CP passaram a representar abordagens complementares para lidar com as limitações de desempenho e eficiência energética dos sistemas modernos. Embora as técnicas de CA ofereçam benefícios expressivos ao permitir que aplicações troquem precisão por desempenho ou eficiência energética, sua adoção eficaz exige identificar cuidadosamente as regiões de código nas quais a aproximação é segura e vantajosa. Essa análise envolve compreender o comportamento da aplicação, seus requisitos de qualidade e o impacto que imprecisões controladas podem ter em seus resultados (Sampson *et al.*, 2015; Reis; Wanner, 2021). Este trabalho parte da hipótese de que é possível combinar CP e CA harmoniosamente, explorando o paralelismo de plataformas modernas enquanto se introduz flexibilidade no nível de precisão. A integração dessas abordagens, especialmente quando apoiada por ferramentas portáteis e amplamente adotadas como o OpenMP, pode ampliar os ganhos de desempenho sem sacrificar a facilidade de uso e a compatibilidade entre diferentes sistemas.

1.1 Objetivos

O objetivo geral deste trabalho é melhorar o desempenho de aplicações de HPC, por meio da combinação de técnicas de aproximação e paralelização.

Para atingir esse objetivo, foram definidos os seguintes objetivos específicos:

- Implementar uma extensão do OpenMP com suporte a técnicas de aproximação baseadas em anotações;
- Desenvolver um conjunto de aplicações voltado à avaliação de técnicas de CA;
- Adicionar suporte em tempo de execução para múltiplos algoritmos de aproximação no OpenMP;
- Avaliar experimentalmente as técnicas propostas em um ambiente de CA.

1.2 Contribuições

As principais contribuições deste trabalho são: a implementação de uma extensão do OpenMP com suporte a técnicas de aproximação e a integração dessas funcionalidades à infraestrutura do LLVM. A extensão proposta exigiu modificações na infraestrutura do LLVM, bem como adaptações em algoritmos e escalonadores do OpenMP, a fim de possibilitar o suporte a tarefas aproximadas. Resultados parciais deste trabalho foram publicados na Escola Regional de Alto Desempenho (ERAD-SP) (Oliveira; Gonçalves; Filho, 2024b), no Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD-2024) (Oliveira; Gonçalves; Filho, 2024a), no Seminário de Iniciação Científica e Tecnológica da UTFPR (SICITE 2024) (Oliveira; Gonçalves; Filho, 2024) e no periódico *Concurrency and Computation: Practice and Experience* (CCPE) (Oliveira; Gonçalves; Filho, 2026).

1.3 Organização do Trabalho

O restante deste trabalho está organizado da seguinte forma: o Capítulo 2 apresenta os conceitos fundamentais de CA e CP relacionados a este estudo, incluindo as técnicas de aproximação abordadas, perfuração de laço, memoização aproximada e relaxamento de ponto flutuante, além de uma introdução ao OpenMP e uma revisão de trabalhos relacionados. Em seguida, o Capítulo 3 descreve a proposta dos construtores implementados no OpenMP utilizando a infraestrutura do LLVM. Já o Capítulo 4 detalha a metodologia, o ambiente e as ferramentas empregadas na realização dos experimentos, bem como as aplicações utilizadas para a aplicação das técnicas. Os resultados obtidos e uma breve discussão são apresentados no Capítulo 5. Por fim, o Capítulo 6 reúne as conclusões do trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos relacionados a CA e CP. Primeiramente aborda-se a definição de CA, seu uso em aplicações, os desafios enfrentados por essa estratégia e o detalhamento dos algoritmos de perfuração de laço, memoização e aproximação de ponto flutuante. Por fim, apresenta também conceitos de CP com um foco no uso da ferramenta OpenMP.

2.1 Computação Aproximada

A CA consiste em um conjunto de técnicas aplicadas a cenários nos quais os resultados exatos não são estritamente necessários ou em que as aplicações apresentam resiliência suficiente para lidar com pequenas imprecisões em suas computações. Nessas situações, o uso de tais técnicas pode proporcionar ganhos significativos de desempenho e eficiência energética, mantendo a margem de erro em níveis reduzidos. Um exemplo é o uso de redes neurais para aproximar a divergência de *branch* em instruções *Single Instruction Multiple Data* (SIMD), capaz de alcançar aceleração de até $14,8\times$ com acurácia de 96% (Grigorian; Reinman, 2015). Por esse motivo, a CA se mostra especialmente atraente em domínios como análise de dados, reconhecimento de padrões, processamento de imagens e sinais (Mittal, 2016; Chippa *et al.*, 2013).

As diferentes estratégias de aproximação podem ser classificadas conforme a camada em que são aplicadas na pilha computacional. No *hardware*, destacam-se técnicas que envolvem desde unidades funcionais aproximadas, como somadores, multiplicadores e divisores, até abordagens como *overclocking* (Leon *et al.*, 2025b; Leon *et al.*, 2025a) e o uso de memórias aproximadas (Fabrício *et al.*, 2020). No *software*, encontram-se estratégias mais generalistas, incluindo linguagens de programação aproximada (Sampson *et al.*, 2015), *runtimes* com suporte à aproximação (Li; Park; Mahlke, 2018; Reis; Wanner; Rigo, 2024) e otimizações realizadas em compilador (Oliveira; Gonçalves; Filho, 2024b; Oliveira; Gonçalves; Filho, 2024a).

Apesar de seus benefícios, essas técnicas enfrentam desafios significativos. Nem todas as aplicações são resilientes a erros em sua execução, limitando o uso de técnicas mais generalistas e torna necessário definir métricas que permitam ajustar o grau de precisão desejado, seja pelo usuário ou pela própria aplicação (Mittal, 2016). Além disso, medir corretamente o nível de erro introduzido não é trivial, pois nem sempre está claro qual métrica deve ser aplicada à saída da aplicação (Felzmann *et al.*, 2021). Outro ponto crítico é a propagação dos erros, que pode comprometer toda a execução: em alguns casos, a aplicação pode falhar abruptamente; em outros, concluir sua execução de forma aparentemente correta, mas com resultados significativamente distantes do esperado (Fabrício *et al.*, 2020).

Diante disso, CA não se restringe a um único método, mas a um conjunto de estratégias que exploram diferentes aspectos da execução de um programa. Na subseção seguinte, são detalhados quatro dessas estratégias: *perfuração de laço*, *memoização* e *a aproximação de ponto flutuante*.

2.1.1 Perforação de Laço

A perforação de laço (*loop perforation*) é uma técnica de computação aproximada que visa reduzir o tempo de execução ao omitir determinadas iterações de um laço (Sidirogloudouskos *et al.*, 2011). Essa abordagem pode ser implementada de forma estática, em tempo de compilação, ou dinâmica, permitindo o ajuste do grau de omissão durante a execução da *runtime* (Li; Park; Mahlke, 2018). Frequentemente, essa otimização é aplicada em laços em formato canônico (Openmp, 2018), como o apresentado no Código 1, que se caracteriza por possuir uma expressão inicial (limite inferior), uma expressão de teste (limite superior) e uma expressão de incremento ou decremento (passo).

Código 1 – Estrutura de um laço canônico.

```
1 for (int i = 0; i < N; i += a) {
2     ...
3 }
```

Fonte: Autoria própria (2026).

Uma forma direta de aplicar a perforação de laço consiste em modificar a expressão de incremento da variável de indução para que parte das iterações seja ignorada. No exemplo do Código 1, se o incremento original for $i++$, todas as iterações são executadas. Se o alterarmos para $i += 2$, o laço passa a saltar uma iteração a cada passo, realizando apenas metade do trabalho computacional. A Figura 1 ilustra o comportamento, dessa modificação nas iterações, na qual os blocos em azul representam as iterações efetivamente executadas e os blocos em branco indicam as iterações omitidas devido ao aumento do passo.

Figura 1 – Modelo de execução do algoritmo de perforação de laço.



Fonte: Autoria própria (2026).

2.1.2 Memoização

A memoização é tradicionalmente uma técnica associada à programação dinâmica que utiliza uma *cache* de resultados para acelerar a execução de determinadas computações. Seu princípio consiste em armazenar os resultados de chamadas já realizadas, indexados pelas entradas correspondentes. Assim, quando a função é invocada novamente com os mesmos parâmetros, o valor previamente calculado é retornado diretamente, evitando a repetição da computação (Michie, 1968).

A memoização aproximada, também chamada de memoização temporal, estende esse conceito ao considerar tempo de execução e localização como critérios adicionais. Sua ideia principal é explorar que chamadas consecutivas a uma mesma função tendem a produzir resultados similares (Tziantzioulis; Hardavellas; Campanoni, 2018).

Na prática, as saídas são armazenadas em uma estrutura de dados e reutilizadas seletivamente em chamadas subsequentes. As estratégias de *cache* podem variar, incluindo modelos globais, específicos por função ou mesmo dependentes de contexto. O reuso é normalmente controlado por um limiar de tolerância, que avalia a variação das saídas: se os resultados permanecerem estáveis dentro desse limite, a função é considerada estável e, portanto, pode ser memoizada (Tziantzioulis; Hardavellas; Campanoni, 2018).

2.1.3 Aproximação de Ponto Flutuante

A representação de números reais por meio de números de ponto flutuante envolvem aproximação, devido à impossibilidade de representar valores contínuos com memória finita (Monniaux, 2008). Por esse motivo, aritmética com números de ponto flutuante usualmente introduz erros de aproximação que variam conforme a ordem de execução dessas operações.

Para reduzir esse tipo de problema, compiladores evitam aplicar otimizações que possam amplificar erros em operações de ponto flutuante. No entanto, alguns compiladores permitem habilitar explicitamente tais otimizações. Por exemplo, Clang e GCC oferecem a opção `-ffast-math`, que permite aplicar essas otimizações durante a compilação de todo o módulo (GNU Project, 2004; The LLVM Project, 2025). O Microsoft Visual C++ (MSVC) disponibiliza a diretiva `#pragma float_control`, que permite definir regiões específicas do código nas quais essas otimizações podem ser aplicadas (Microsoft, 2021). Por fim, o compilador da Intel (ICC) também fornece suporte a aproximações em ponto flutuante por meio das *flags* `fp-speculation`, `Qfp-speculation` e `-fp-model=fast [=1|2]` (Intel, 2026).

2.2 Impacto da Aproximação no Resultado de Computações

A aplicação de técnicas de CA introduz imprecisões nos resultados das computações. Essas imprecisões podem causar diferentes impactos no sistema, variando desde erros silenciosos até falhas que interrompem a execução da aplicação (Fabrício, 2022). Grande parte das pesquisas em computação aproximada concentra-se no controle dos erros silenciosos, pois, apesar da perda de precisão, o sistema ainda é capaz de produzir resultados úteis. Em contrapartida, falhas diretas comprometem o fluxo de execução e impedem a geração de saídas válidas, como ocorre em acessos inválidos à memória (*segmentation faults*).

A CA busca estabelecer um equilíbrio entre o nível de aproximação aplicado e as falhas resultantes, de modo que o usuário final sofra o menor impacto possível na precisão dos resultados, enquanto se obtém um ganho significativo no uso de recursos computacionais. Entretanto, um dos desafios enfrentados por essas técnicas está na dificuldade de caracterizar como as aproximações afetam o resultado, especialmente quando se consideram aplicações com naturezas e requisitos de precisão distintos.

Nesta pesquisa, utilizam-se métricas de qualidade para quantificar a divergência entre as saídas aproximadas e as saídas exatas, com uma seleção fundamentada na literatura e

nas especificidades de cada aplicação. Embora esses indicadores forneçam uma base objetiva para a comparação sistemática de diferentes níveis de aproximação, os valores numéricos nem sempre refletem a utilidade prática do resultado. Conforme observado por Felzmann *et al.* (2021), uma saída com métricas favoráveis pode ser irrelevante para o propósito final do sistema, enquanto resultados com menor precisão, como uma imagem ruidosa que ainda permite o reconhecimento de objetos, podem permanecer aceitáveis. Assim, embora a análise exclusiva por métricas quantitativas possa induzir a interpretações imprecisas sobre a utilidade real dos dados, elas permanecem fundamentais como critério de controle e comparação sistemática.

Para avaliar a perda de acurácia das aplicações, foram adotadas três métricas distintas: o *Mean Absolute Percentage Error* (MAPE), o *Miss-Classification Rate* (MCR) e o *Structural Similarity Index Measure* (SSIM), definidos, respectivamente, nas Equações 1, 2 e 3.

$$MAPE(A, F) = \frac{100}{n} \sum_{t=1}^n \left| \frac{A_t - F_t}{A_t} \right| \quad (1)$$

$$MCR(A, F) = \frac{1}{n} \sum_{t=1}^n \iota(A_t \neq F_t) \quad (2)$$

Nas Equações 1 e 2 as variáveis são definidas por:

- n : número de elementos no *array* de dados;
- A_t : t -ésimo elemento obtido pela execução acurada;
- F_t : t -ésimo elemento obtido pela execução aproximada;
- ι : função indicadora que retorna 1 quando a condição é verdadeira e 0 caso contrário.

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + c_1) \cdot (2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1) \cdot (\sigma_x^2 + \sigma_y^2 + c_2)} \quad (3)$$

Já na Equação 3 as variáveis são definidas por:

- μ_x : média dos *pixels* da imagem x ;
- μ_y : média dos *pixels* da imagem y ;
- σ_x : variância dos *pixels* da imagem x ;
- σ_y : variância dos *pixels* da imagem y ;
- σ_{xy} : covariância entre as imagens x e y ;
- c_1 e c_2 : constantes pequenas utilizadas para evitar divisão por zero.

2.3 Computação Paralela

A CP é uma área da computação que busca explorar arquiteturas modernas, cada vez mais baseadas em processadores *multicore*. Esse conceito pode ser melhor compreendido a partir da taxonomia de Flynn (Flynn, 1972), que classifica as arquiteturas de computadores conforme o fluxo de instruções e de dados:

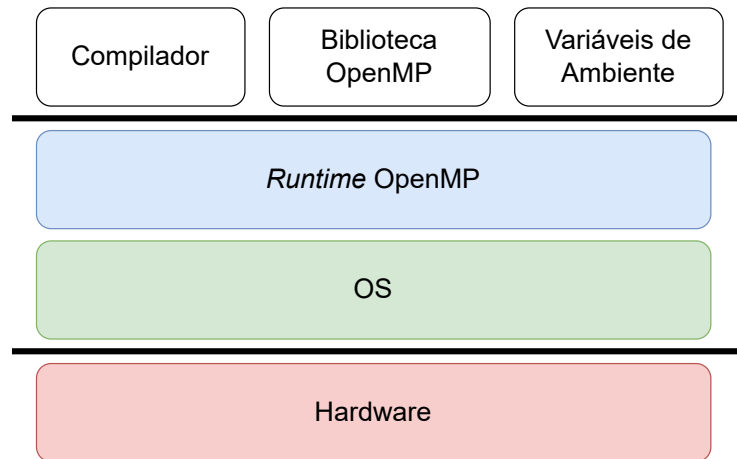
- **Single Instruction, Single Data stream (SISD):** Instrução única que atua sobre uma única fonte de dados.
- **Single Instruction, Multiple Data stream (SIMD):** Instrução única aplicada simultaneamente a múltiplos dados.
- **Multiple Instructions, Single Data stream (MISD):** Múltiplas instruções atuando sobre uma mesma fonte de dados.
- **Multiple Instructions, Multiple Data stream (MIMD):** Múltiplas instruções atuando em paralelo sobre múltiplas fontes de dados.

Dentro desse modelo, o paralelismo pode ser explorado de diferentes maneiras. No nível do processador, técnicas como *pipelining*, predição de desvios (*branch prediction*) e escalonamento dinâmico permitem dividir o código em pequenas unidades que podem ser executadas simultaneamente.

Algumas arquiteturas foram além e incorporaram extensões SIMD, permitindo a utilização de processadores vetoriais capazes de realizar a mesma operação sobre múltiplos elementos de um vetor unidimensional em um único ciclo de instrução. Em cenários mais especializados, como nas *Graphics Processing Units* (GPUs), esse modelo é expandido para múltiplas dimensões de dados, possibilitando que milhares de operações sejam executadas em paralelo e tornando-as adequadas para cargas de trabalho massivamente paralelizáveis (Hennessy; Patterson, 2017). Outro mecanismo fundamental é o uso de *threads*, que representam abstrações de execução ao nível de software, permitindo que diferentes unidades computacionais operem concorrentemente em múltiplos processadores. Cada *thread* possui seu próprio contexto de execução, mas pode compartilhar recursos como memória e dispositivos de entrada/saída, viabilizando desde o paralelismo de tarefas independentes até a cooperação em algoritmos que exploram granularidades mais finas (Tanenbaum, 2015; Hennessy; Patterson, 2017).

É nesse contexto que surge o OpenMP (OpenMP Architecture Review Board, 2021), uma *Application Programming Interface* (API) projetada para estender C, C++ e Fortran com suporte à paralelização de aplicações por meio de anotações de código. Essas anotações especificam o como aquela região de código deve ser executada. Seu modelo de programação permite explorar automaticamente diferentes formas de paralelismo, seja pelo *offloading* para GPUs, pelo uso de múltiplas *threads*, ou ainda pela vetorização via SIMD, aproveitando as unidades vetoriais das arquiteturas modernas (Mattson; He; Koniges, 2019).

Figura 2 – Arquitetura do OpenMP.



Fonte: Autoria própria (2026).

O OpenMP é estruturado em uma arquitetura em camadas, composta por anotações de código, suporte da *runtime* e configurações do ambiente de execução, conforme ilustrado na Figura 2. Esse modelo permite ao programador controlar diversos aspectos da execução por meio de diretivas e chamadas à biblioteca, como a definição do número de *threads* em uma região paralela e a escolha do comportamento do escalonador. Dessa forma, o OpenMP oferece um controle flexível sobre o paralelismo do programa.

Código 2 – Implementação sequencial de um produto escalar.

```
1 for (int i = 0; i < N; i++) {
2     result += a[i] * b[i];
3 }
```

Fonte: Autoria própria (2026).

O Código 2 apresenta a implementação sequencial de um produto escalar entre vetores. A paralelização utilizando a biblioteca `pthread` (Padua, 2011), apresentado no Código 3, exige a criação explícita de *threads*, gerenciamento de regiões de memória e sincronização dos resultados parciais. Em contraste, a Código 4 implementado com o OpenMP, evidencia a simplicidade proporcionada pelas diretivas: uma única anotação é suficiente para paralelizar o laço de cálculo, delegando à *runtime* a criação e sincronização das *threads*.

Tipicamente, uma diretiva OpenMP é aplicada a uma região de código, como um laço, para indicar que pode ser executada em paralelo. Durante a compilação, o compilador isola o bloco de código anotado e o transforma em uma função separada, conhecida como *outlined function*, na qual variáveis privadas, memória de pilha e pontos de sincronização são automaticamente configurados. Em tempo de execução, a *runtime* do OpenMP é responsável por gerenciar essas funções extraídas, criando e coordenando múltiplas *threads* que executam cópias dessa função segundo o modelo *fork-join*. Por exemplo, a diretiva `#pragma omp parallel` cria um time de *threads*, cada uma executando a função correspondente à região paralela. Ao final da região paralela, as *threads* são sincronizadas (*join*) e a execução retorna ao fluxo se-

Código 3 – Implementação paralelizada com pthreads de um produto escalar.

```

1 void *compute_partial_dot(void *arg) {
2     ThreadArgs *args = (ThreadArgs *)arg;
3     args->partial_sum = 0.0;
4
5     for (int i = args->start; i < args->end; i++) {
6         args->partial_sum += args->a[i] * args->b[i];
7     }
8
9     return NULL;
10 }
11
12 ...
13
14 pthread_t threads[NUM_THREADS];
15 ThreadArgs args[NUM_THREADS];
16 result = 0.0;
17 chunk_size = N / NUM_THREADS;
18
19 for (int i = 0; i < NUM_THREADS; i++) {
20     args[i].start = i * chunk_size;
21     args[i].end = (i == NUM_THREADS - 1) ? N : (i + 1) *
        chunk_size;
22     args[i].a = a;
23     args[i].b = b;
24     pthread_create(&threads[i], NULL, compute_partial_dot,
        &args[i]);
25 }
26
27 for (int i = 0; i < NUM_THREADS; i++) {
28     pthread_join(threads[i], NULL);
29     result += args[i].partial_sum;
30 }

```

Fonte: Autoria própria (2026).

quencial. Essa abordagem permite paralelizar regiões de código com esforço reduzido de gerenciamento, mantendo controle sobre sincronização e compartilhamento de dados (Mattson; He; Koniges, 2019).

Código 4 – Implementação paralelizada com OpenMP de um produto escalar.

```

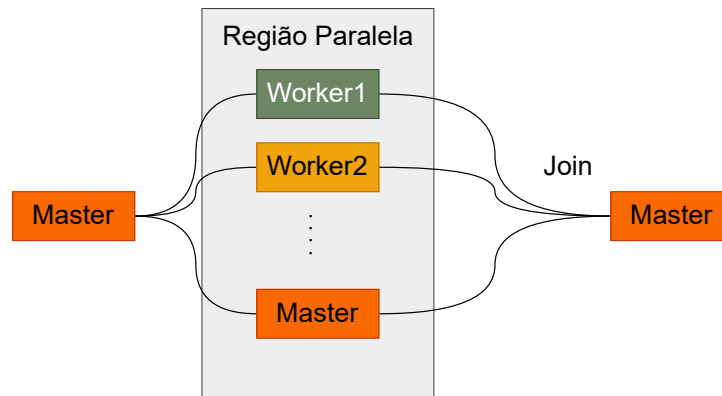
1 #pragma omp parallel for reduction(+ : result)
2 for (int i = 0; i < N; i++) {
3     result += a[i] * b[i];
4 }

```

Fonte: Autoria própria (2026).

O modelo *fork-join* pode ser observado de forma esquemática na Figura 3. Nele, a *run-time* cria múltiplas *threads* para a execução paralela, sincronizadas ao final, restando apenas a *thread* principal para continuar a execução sequencial. O OpenMP também suporta regiões paralelas aninhadas, possibilitando hierarquias de paralelismo.

Figura 3 – Modelo *fork-join* de execução do OpenMP.



Fonte: Autoria própria (2026).

2.4 Trabalhos Relacionados

Esta seção apresenta arcabouços que utilizam técnicas de CA, implementadas em nível de linguagem de programação, especialmente aqueles que utilizam anotações de código e mecanismos de execução em *runtime*. Os arcabouços e aplicações selecionados buscam oferecer ao desenvolvedor maior controle sobre as técnicas utilizadas e sobre as regiões de código que podem ser aproximadas.

O Approximate C Compiler for Energy and Performance Trade-Off (ACCEPT) (Sampson *et al.*, 2015) é um arcabouço implementado sobre a infraestrutura do LLVM, que introduz a palavra-chave ACCEPT na sintaxe das linguagens C/C++. Essa palavra-chave permite diferenciar regiões de código aproximadas daquelas que devem ser executadas de forma precisa. O arcabouço também implementa etapas de análise para verificar a possibilidade de aproximação dessas regiões, avaliando a validade das transformações, e viabilizando a aplicação de diferentes técnicas de otimização aproximada, de maneira análoga ao funcionamento de um compilador paralelizador.

A infraestrutura do ACCEPT permite, por meio de anotações de código e análises estáticas e dinâmicas, identificar regiões de código potencialmente aproximáveis. Essas regiões podem aplicar técnicas como perfuração de laço, remoção de estruturas de sincronização e até mesmo o uso de redes neurais em *hardware* para estimar saídas. Além disso, o arcabouço incorpora um *autotuner* que utiliza métricas fornecidas pelo usuário para avaliar se as aproximações aplicadas mantêm a qualidade de saída nos limites esperados, garantindo que o programa não apresente falhas durante a execução.

Vassiliadis *et al.* (2015) propõem uma extensão ao OpenMP que incorpora suporte à CA no modelo de programação baseado em tarefas. Em sua proposta, o programador pode especificar funções aproximadas alternativas que podem ser escolhidas em tempo de execução para substituir versões precisas de determinadas tarefas. Assim, parte das tarefas é executada de forma exata, enquanto outras utilizam versões aproximadas, reduzindo o consumo energético e mantendo a qualidade de acordo com limites definidos pelo usuário.

O arcabouço também introduz um mecanismo de barreira para garantir que uma fração mínima das execuções alcance o grau de acurácia desejado. Para gerenciar essa execução, são propostas duas políticas no *runtime*: *Global Task Buffering* (GTB), que toma decisões informadas a partir da análise conjunta de todas as tarefas pendentes, e a *Local Queue History* (LQH), baseada no histórico local de filas, em que cada *worker* decide de maneira autônoma o nível de aproximação com base nas execuções anteriores.

Lashgar, Atoofian e Baniasadi (2018) propõem a integração da técnica de perfuração de laço ao OpenACC (Organization, 2020), um padrão de diretivas para paralelização e aceleração de aplicações em dispositivos como GPUs. Assim como no OpenMP, o OpenACC utiliza anotações para indicar regiões do código que podem ser executadas em paralelo ou transferidas para aceleradores. Nesse contexto, os autores aplicam perfuração de laço para melhorar o desempenho, executando apenas um subconjunto das iterações do laço. Os autores introduzem um mecanismo de correção implementado diretamente no kernel que identifica as saídas ausentes e aplica estratégias como a cópia ou a média dos valores vizinhos já computados, a fim de estimar as entradas faltantes.

O HPAC (Parasyris *et al.*, 2021) é um arcabouço de técnicas de computação aproximada voltadas para aplicações HPC, implementado sobre a infraestrutura do LLVM. Ele utiliza anotações de código `pragma` para C/C++, permitindo a definição de regiões de código aproximadas, e se destaca por sua interoperabilidade com o OpenMP. As técnicas principais incluem perfuração de laço e memoização aproximada, e o arcabouço fornece ainda uma ferramenta de análise que permite ao usuário anotar múltiplas regiões de código e avaliar a qualidade dos resultados obtidos para cada configuração.

O HPAC-Offload (Fink *et al.*, 2023) é uma extensão do HPAC (Parasyris *et al.*, 2021) voltada para aplicações em GPU, que adapta sua arquitetura para lidar com as limitações do modelo de memória e paralelismo dessas plataformas. Assim como no HPAC, as técnicas implementadas incluem perfuração de laço e memoização aproximada, mas com um gerenciamento de memória consciente da GPU: os dados internos são armazenados na memória compartilhada do bloco, permitindo reutilização entre threads ativas sem sobrecarregar a memória global. O arcabouço também define níveis hierárquicos de aproximação (*thread*, *warp* e bloco) para evitar divergência e *deadlocks*.

Em contraste com outros trabalhos, nossa abordagem baseia-se na extensão da implementação do OpenMP utilizada pelo LLVM, cuja *runtime* deriva da implementação originalmente desenvolvida pela Intel (Llvm, 2026). Para isso, modificamos a *runtime* e estendemos suas diretivas de compilação para oferecer suporte nativo a técnicas de CA. Ao incorporar o controle de aproximação diretamente no OpenMP, habilitamos o escalonamento e a execução de tarefas sem comprometer sua API. Essa integração simplifica a adoção de CA em CP, como também abre caminho para uma exploração mais ampla e eficaz da aproximação em aplicações que já utilizam o OpenMP. A Tabela 1 apresenta um breve resumo e comparação entre todos os trabalhos citados.

Tabela 1 – Resumo e comparação entre os trabalhos relacionados.

Trabalho	Linguagem / Infraestrutura	Suporte a Paralelismo	Técnicas de Aproximação	Diferenciais
ACCEPT (Sampson <i>et al.</i> , 2015)	C/C++, LLVM	Sequencial e paralelo	Perforação de laço, elisão de sincronização e redes neurais em <i>hardware</i>	Validação de aproximações em tempo de execução; Aplicação de múltiplas técnicas de otimização
Vassiliadis <i>et al.</i> (Vassiliadis <i>et al.</i> , 2015)	C/C++, OpenMP	Paralelismo baseado em tarefas	Funções aproximadas	Controle de acurácia; Decisões globais e locais para aproximação
Lashgar <i>et al.</i> (Lashgar; Atoofian; Baniasadi, 2018)	C/C++, OpenACC	Paralelismo em laço	Perforação de laço	Estima valores ausentes para manter precisão em laços aproximados
HPAC (Parasyris <i>et al.</i> , 2021)	C/C++, LLVM, OpenMP	Regiões paralelas	Perforação de laço, memoização aproximada	Interoperabilidade com OpenMP; Análise de múltiplas regiões de código
HPAC-Offload (Fink <i>et al.</i> , 2023)	C/C++, LLVM, OpenMP	Paralelismo via GPU	Perforação de laço, memoização aproximada	Interoperabilidade com OpenMP; Hierarquia de aproximação em GPU
Este Trabalho	C/C++, LLVM, OpenMP	Regiões paralelas	Perforação de laço, memoização aproximada, aproximação em ponto flutuante	Integração direta com OpenMP; Combina análise em tempo de compilação e decisões em tempo de execução; Aplica múltiplas técnicas de aproximação visando desempenho

Fonte: Autoria própria (2026).

3 PROPOSTA

Este trabalho propõe o desenvolvimento de um novo construtor para a API OpenMP, visando facilitar a implementação de algoritmos de CA. Alinhada ao modelo de execução baseado em diretivas de OpenMP, a proposta utiliza diretivas de compilação para identificar regiões passíveis de execução aproximada. A integração ao OpenMP assegura que tais técnicas sejam portáteis e adaptáveis a diferentes trechos de código. Neste contexto, uma região aproximada corresponde ao bloco imediatamente associado à diretiva, que por sua vez é delimitado pelas chaves `{` e `}`, que definem o escopo no qual as técnicas de aproximação serão aplicadas.

O Código 5 apresenta a sintaxe da diretiva `#pragma omp approx`, que delimita uma região de código na qual a técnica de aproximação será aplicada. Embora o construtor permita a combinação de múltiplas cláusulas para a definição de parâmetros, a implementação atual suporta o uso de uma técnica por região de aproximada. Essa escolha de projeto visa garantir uma avaliação isolada de cada abordagem, facilitando a análise comparativa e evitando a explosão combinatória dos experimentos.

Código 5 – Sintaxe do construtor `approx`.

```
1 #pragma omp approx {memo(<option>), perfo(<option>),
   fastmath} [[,] clause]
2
```

Fonte: Autoria própria (2026).

Além da diretiva, a proposta introduz a variável de ambiente `OMP_APPROX`, que permite habilitar (`TRUE`) ou desabilitar (`FALSE`) as técnicas de aproximação sem a necessidade de modificar o código-fonte ou realizar novas compilações. A implementação desse sistema assemelha-se ao funcionamento da cláusula `if`. Na prática, o compilador gera tanto a versão aproximada quanto a versão original do código. Durante a execução, o valor da variável de ambiente é verificado para decidir qual caminho de execução deve ser seguido, caso ela não esteja configurada o código original é executado.

As seções subsequentes detalham as técnicas implementadas, suas respectivas cláusulas e parâmetros de configuração.

3.1 Perforação de Laço

A cláusula `perfo` habilita o uso de perforação de laços (*loop perforation*), permitindo reduzir o número de iterações executadas. Essa cláusula deve ser utilizada em conjunto com a cláusula `for`, e suporta modificadores que definem tanto o algoritmo de perforação quanto a quantidade de iterações a serem descartadas. Os modificadores disponíveis são: `init`, `fini` e `large`. O Código 6 apresenta a sintaxe da cláusula, na qual é necessário especificar um dos tipos de perforação e a taxa de descarte, e no Código 7 temos um exemplo de uso dessa diretiva junto a cláusula `for`.

Código 6 – Sintaxe da cláusula `perfo`.

```

1 #pragma omp approx perfo({init, fini, large}, drop-rate)
2

```

Fonte: Autoria própria (2026).

Código 7 – Exemplo de uso da cláusula `perfo`.

```

1 #pragma omp for approx perfo(init, 10)
2 for (int i = 0; i < N; i++) {
3     ...
4 }
5

```

Fonte: Autoria própria (2026).

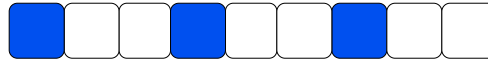
O modificador `init` descarta iterações no início do laço. A implementação dessa técnica depende do escalonador da *runtime*. No OpenMP, o escalonador distribui o espaço de iterações entre as *threads* por meio da cláusula `schedule(type [, chunk])`. Durante a execução paralela, parâmetros como o limite inferior, limite superior e o tamanho do passo são fornecidos a cada *thread* para o processamento de sua carga de trabalho. A técnica proposta atua sobre o limite inferior, incrementando seu valor original para que a iteração inicie em um ponto avançado do laço.

Essa abordagem apresenta variações de comportamento conforme o modificador da cláusula `schedule`. No escalonamento `static`, as iterações são divididas em blocos de tamanho fixo (*chunks*) e a distribuição segue um modelo estático, isso garante que cada tarefa processe um volume reduzido de iterações de forma previsível. Em contrapartida, nos escalonamentos `dynamic` e `guided`, as *threads* solicitam novos blocos de iterações sob demanda ao finalizarem suas tarefas anteriores. Nesses casos, a peroração torna-se imprevisível, pois a carga de trabalho total processada individualmente depende da velocidade de execução e da latência de sincronização, o que pode resultar em descartes inferiores aos observados no modelo estático. O comportamento desse modificador é apresentado na Figura 4a, na qual os quadros azuis representam as iterações executadas e os quadros brancos indicam as iterações não executadas.

O modificador `fini` também interage com o escalonador para realizar o descarte de iterações. Sua diferença para o `init` é a mudança no limite superior recebido pelas *threads*. O limite é reduzido com base no valor especificado para o descarte, impedindo que as iterações alcancem o final original da execução. O `fini` apresenta as mesmas limitações do `init` em relação aos tipos de escalonadores. A Figura 4b apresenta o modelo de execução deste modificador.

Por fim, o modificador `large` difere das técnicas anteriores por ser implementado diretamente no processo de geração de código. Com base na taxa de descarte especificada, o compilador gera um incremento adicional no laço para que as iterações realizem saltos maiores. Por ser um modificador que altera diretamente o código, essa técnica não sofre variações conforme os diferentes tipos de escalonador. A Figura 4c ilustra esse comportamento.

Figura 4 – Características de cada modificador.

(a) Comportamento do modificador `init`.(b) Comportamento do modificador `fini`.(c) Comportamento do modificador `large`.

Fonte: Autoria própria (2026).

3.2 Aproximação de Ponto Flutuante

A cláusula `fastmath` introduz otimizações agressivas de ponto flutuante em regiões específicas do código. Diferentemente da *flag* de compilação `-ffast-math` do Clang (The LLVM Project, 2025), que atua globalmente sobre todo o módulo, essa cláusula oferece um controle granular ao restringir as otimizações aos blocos delimitados pela diretiva `#pragma omp approx fastmath`. Assim, trechos de código podem se beneficiar de ganhos de desempenho sem comprometer a acurácia de trechos críticos do restante do programa. Além disso, ao contrário da cláusula `perfo`, a `fastmath` não se limita a laços de repetição, podendo ser aplicada a blocos arbitrários de código.

Quando a cláusula é utilizada, o compilador passa a habilitar as mesmas otimizações associadas à `-ffast-math`, flexibilizando operações de ponto flutuante, permitindo reordenação de operações aritméticas e eliminando verificações de segurança que impactam o desempenho. Além disso, a cláusula integra uma biblioteca matemática aproximada desenvolvida pelo próprio autor especificamente para este projeto. A sintaxe da diretiva é apresentada no Código 8.

Código 8 – Sintaxe da cláusula `fastmath`.

```
1 #pragma omp approx fastmath
```

Fonte: Autoria própria (2026).

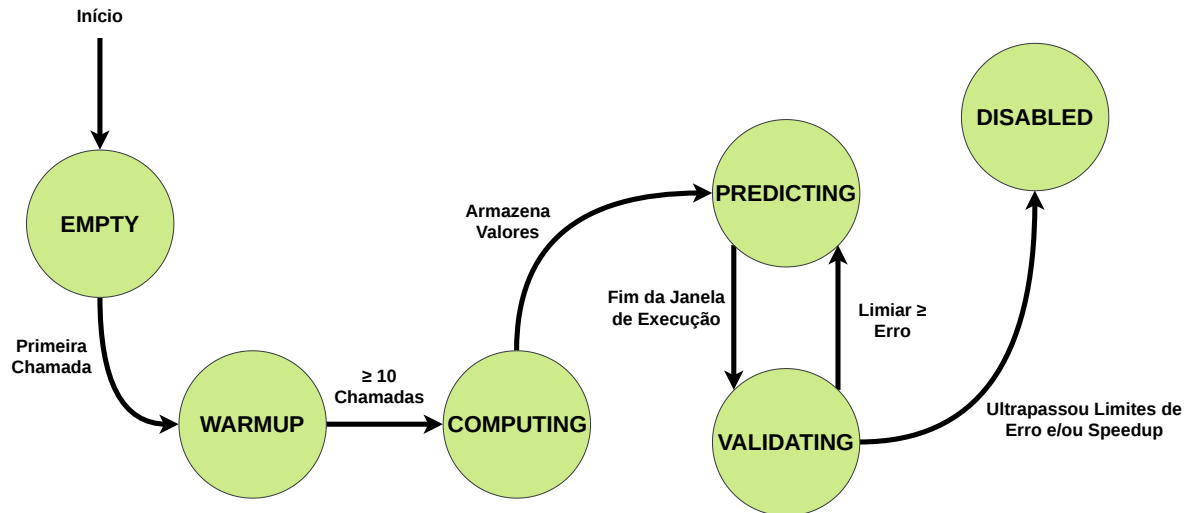
O Código 9 apresenta um exemplo de uso da cláusula. Ao aplicá-la, o compilador habilita otimizações agressivas de ponto flutuante para a região marcada.

Código 9 – Exemplo de aplicação da cláusula `fastmath`.

```
1 void kernel(float *a, float *b, int n) {
2     #pragma omp approx fastmath
3     for (int i = 0; i < n; i++) {
4         a[i] = exp(a[i]) + log(b[i]) * 0.5f;
5     }
6 }
```

Fonte: Autoria própria (2026).

Figura 5 – Diagrama de estados da cláusula *memo*.



Fonte: Autoria própria (2026).

3.3 Memoização

A cláusula *memo* habilita a memoização aproximada em blocos de código arbitrários. Essa estratégia explora regiões que apresentam relações de entrada e saída estáveis, nas quais execuções sucessivas produzem resultados consistentes. O Código 10 apresenta a sintaxe da cláusula. A cláusula `memo` requer o parâmetro *threshold*, definido entre 0 e 100, que estabelece o limite de erro percentual tolerável das amostragens coletadas para com o resultado original da região. Além disso, seu uso exige a cláusula *output*, que especifica uma única variável de saída a ser aproximada.

Código 10 – Sintaxe da cláusula *memo*.

```
1 #pragma omp approx memo(threshold) output(variable)
```

Fonte: Autoria própria (2026).

Internamente, a técnica utiliza uma estrutura de tabela *hash* local a cada *thread*, denominada `kmp_map_t`. Essa estrutura armazena todas as regiões de código anotadas com a cláusula *memo*. Cada região é identificada por uma chave inteira e associada a uma estrutura interna que mantém o estado da execução, o histórico de valores observados, as previsões geradas e métricas de desempenho utilizadas para controle adaptativo. Essa organização permite que a *runtime* intercepte a região anotada e gerencie o ciclo de vida dos dados sem contenção entre *threads*.

O funcionamento baseia-se em uma máquina de estados que controla o processo de aprendizado e validação da aproximação. Os estados utilizados são *empty*, *warmup*, *computing*, *predicting*, *validating* e *disabled*. Cada estado define uma etapa específica do ciclo de execução da memoização, a Figura 5 apresenta uma ilustração das transições entre esses estados.

Inicialmente, a região encontra-se no estado *empty*, indicando que ainda não há dados associados àquela entrada na tabela. Em seguida, o sistema transita para o estado *warmup*. Nesse estágio, o código original é executado normalmente por um número fixo de chamadas, atualmente configurado para 10 execuções. O objetivo é coletar uma referência inicial de tempo de execução da região, que será utilizada posteriormente para avaliar o custo adicional introduzido pela memoização.

Após a fase de *warmup*, o sistema entra no estado *computing*. Nessa etapa, o código original continua sendo executado e os valores produzidos são armazenados em um histórico associado à entrada da tabela. Esse histórico constitui a base de dados utilizada para gerar previsões futuras.

Quando dados suficientes forem coletados, o sistema transita para o estado *predicting*. Nesse estágio, a *runtime* deixa de executar o bloco de código original e passa a fornecer diretamente um valor previsto. A quantidade de previsões consecutivas é controlada por uma janela dinâmica. Essa janela define quantas execuções podem ser aproximadas antes que o sistema realize uma nova verificação do modelo.

Ao atingir o limite da janela, o sistema entra no estado *validating*. Nesse momento, o código original é executado novamente e o resultado real é comparado ao valor previsto. O erro é calculado conforme a métrica MAPE. Se o erro estiver dentro do limite estabelecido pelo *threshold*, o tamanho da janela de previsão é ampliado de forma exponencial, dobrando seu valor até o limite máximo de 2^{20} . Caso o erro ultrapasse o limite permitido, a janela é reduzida para uma unidade e o novo valor real é inserido no histórico. Esse mecanismo permite que o modelo se adapte a mudanças no comportamento da aplicação ao longo do tempo.

Se o número de falhas consecutivas ultrapassar um limite calculado a partir do *threshold*, a memoização é desabilitada permanentemente para aquela região. Esse limite é descrito pela Equação 4.

$$T = \frac{100}{\text{threshold} + 1} \quad (4)$$

Além da validação baseada em erro, o sistema também monitora o custo de execução da memoização. A cada intervalo de chamadas, o tempo médio da execução aproximada é comparado ao tempo médio observado durante a fase de aquecimento. Se o custo adicional exceder um fator de tolerância predefinido, a região é colocada no estado *disabled*. Essa verificação evita que a técnica introduza *overhead* significativo e degrade o desempenho da aplicação.

A geração das previsões é realizada por meio de uma média aritmética aplicada aos valores armazenados no histórico do *bucket*. Esse processo funciona como uma média móvel, na qual cada novo valor observado é inserido em uma janela circular de tamanho limitado. A Equação 5 apresenta a expressão utilizada para calcular o valor previsto, na qual x_i representa cada valor normalizado armazenado e N corresponde ao número de elementos disponíveis no histórico.

$$P = \frac{1}{N} \sum_{i=1}^N x_i \quad (5)$$

Para variáveis booleanas, o sistema aplica um limiar simples: o valor previsto é considerado verdadeiro quando a média calculada é maior ou igual a 0,5. Esse comportamento permite tratar dados discretos mantendo consistência com a lógica binária da aplicação.

O Código 11 demonstra a aplicação da cláusula em um laço de repetição. Nesse exemplo, a variável *sum* é memoizada com um *threshold* de erro de 10%.

Código 11 – Exemplo de uso da cláusula *memo*.

```
1 for (int i = 0; i < N; i++) {  
2     #pragma omp approx memo(10) output(sum)  
3     {  
4         sum += values[i];  
5     }  
6 }
```

Fonte: Autoria própria (2026).

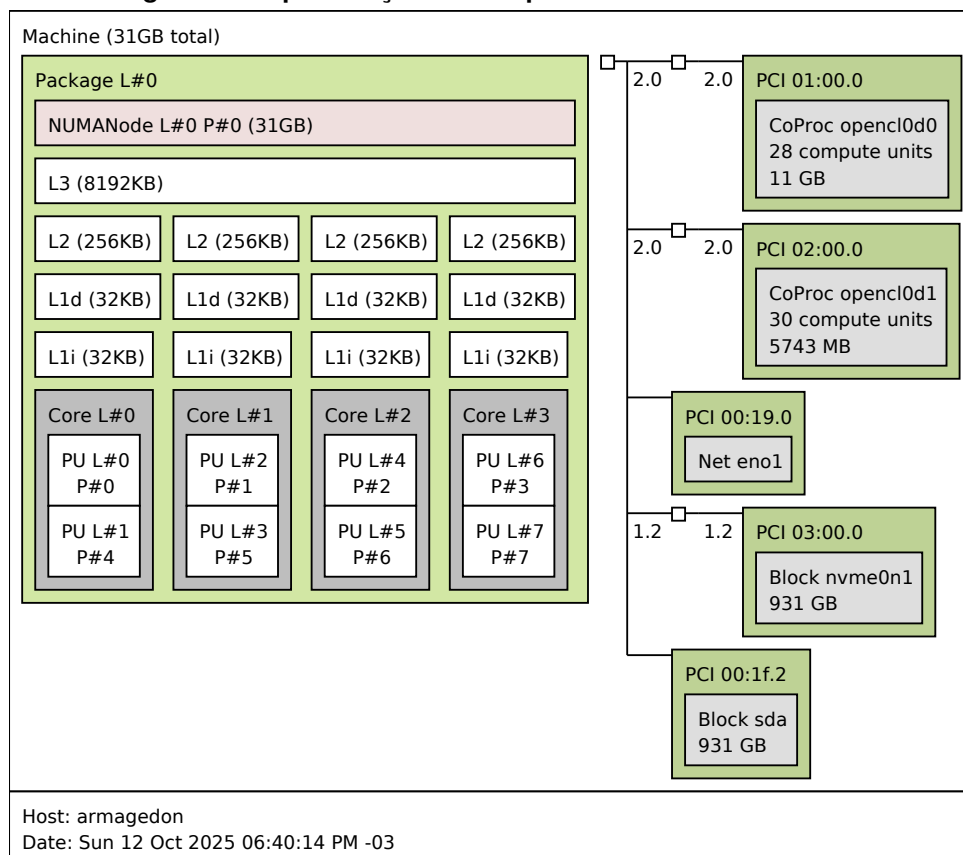
4 MATERIAIS E MÉTODOS

Este capítulo apresenta a metodologia e os recursos utilizados no desenvolvimento deste trabalho. A Seção 4.1 descreve as configurações de *hardware* das máquinas empregadas na execução dos experimentos. Em seguida, a Seção 4.2 detalha as aplicações selecionadas para os testes, bem como as técnicas de otimização adotadas e o posicionamento das regiões paralelas. Por fim, a Seção 4.3 apresenta uma síntese de todos os experimentos realizados, organizada em formato tabular.

4.1 Ambiente e Configuração Experimental

Os experimentos foram conduzidos em um computador com processador Intel Core i7-4790, operando a uma frequência base de 3,6 GHz. O processador possui quatro núcleos físicos e suporte a oito *threads*. A hierarquia de memória é composta por 32 KB em L1, 256 KB em L2 e 8 MB em L3, além de 32 GB de memória RAM DDR3, divididos em quatro módulos de 8 GB em *dual-channel* a 1600 MT/s. O sistema operacional adotado foi o Ubuntu 22.04.4 LTS. A Figura 6 detalha a arquitetura de memória e a organização dos núcleos utilizados nos testes.

Figura 6 – Especificação do computador utilizado em testes.



Fonte: Autoria própria (2026).

As aplicações foram executadas com 1, 2, 4 e 8 *threads*. Cada configuração executada 10 vezes para mitigar o impacto de anomalias de execução. O ambiente foi configurado com

a variável `OMP_PROC_BIND="TRUE"` para fixar as *threads* aos núcleos físicos e `OMP_APPROX="TRUE"` para ativar as rotinas de computação aproximada. Regiões que utilizaram da cláusula `for` foram executadas com o escalonamento estático. Algoritmos dependentes de números aleatórios utilizaram sementes fixas, o que garante a reprodutibilidade dos dados.

O conjunto de *benchmarks* foi testado isoladamente com cinco técnicas de aproximação: `fastmath`, `perfo_init`, `perfo_fini`, `perfo_large` e `memo`, as aplicações que foram utilizadas estão disponíveis no repositório do projeto ¹. A técnica `perfo_large` foi testado com uma taxa de descarte de 10, 20, 30, 40 e 50. A `perfo_init` e `perfo_fini` foram avaliadas com um nível de descarte de 10, 100, 500 e 1000. Cabe ressaltar que os parâmetros fornecidos para as abordagens `perfo` não representam porcentagens, mas sim valores absolutos que indicam a quantidade exata de iterações do laço que serão descartadas. Já a técnica `memo` operou com taxas de 10, 25, 50 e 100. A escolha desses parâmetros teve como objetivo representar diferentes níveis de agressividade para cada técnica, permitindo observar o comportamento e a degradação das aplicações sob variadas intensidades de aproximação. A diretiva `fastmath` não possui parâmetros de configuração. Todos os códigos foram compilados com a *flag* `-O3` para maximizar o desempenho base. Como as modificações de aproximação acontecem apenas na fase de geração de código ou em tempo de execução, as otimizações do compilador não interferem na semântica das técnicas avaliadas, o que preserva o comportamento esperado.

A medição do desempenho das aplicações foi realizada com base no tempo total de execução, utilizando a ferramenta `perf` para a coleta dessas informações. A versão exata do algoritmo, executada com `OpenMP` e uma única *thread*, foi utilizada como *baseline* tanto para as métricas de desempenho quanto para as métricas de qualidade, estas últimas calculadas a partir da comparação com a execução sem o uso de diretivas de aproximação.

Além das medições de tempo, a identificação das regiões quentes de código foi conduzida com o auxílio do `valgrind`, por meio da análise da quantidade de ciclos consumidos em diferentes trechos da aplicação. Embora ciclos de processador representem uma métrica mais detalhada do custo computacional interno, a utilização do `perf` em aplicações paralelas não fornece uma maneira simples de identificar o consumo de ciclos individual de cada *thread*, especialmente considerando que os ciclos são acumulados entre elas. Dessa forma, o tempo de execução foi adotado como principal métrica de desempenho para a comparação entre as versões das aplicações, enquanto a análise de regiões quentes utilizou ciclos de processador para identificar regiões de maior custo computacional.

4.2 Conjunto de *Benchmarks*

A seleção das aplicações que compõem o conjunto de *benchmarks* teve como principais critérios a tolerância a erros e a estrutura do algoritmo. Buscou-se contemplar uma gama ampla de domínios, incluindo *machine learning*, álgebra linear, modelos estatísticos e multimídia.

¹ Disponível em: <https://github.com/Victor-Briganti/approx-benchmark>

Outro objetivo foi preservar ao máximo o código original durante a inserção das regiões paralelizadas, evitando modificações significativas na implementação. As aplicações *2MM*, *Correlação*, *Deriche* e *Jacobi 2D* foram obtidas a partir do conjunto PolyBench (Pouchet, 2011), K-Means foi obtida a partir do Rodinia (Che *et al.*, 2009) e o Mandelbrot e PI de Monte Carlo foram adaptações do Debian Benchmarks Game (Debian Project, 2018). Em relação à precisão dos dados, o tipo padrão adotado foi o `double`, com exceção do *Deriche* e do *K-Means*, que operam com `float`. Esse tipo padrão foi definido, para submeter as aplicações a um ambiente de alta carga computacional.

4.2.1 2MM

O *benchmark* 2MM (Pouchet, 2011) realiza a multiplicação sequencial de matrizes utilizando três matrizes distintas: A, B e C. O processo de execução é ilustrado pelas Equações 6 e 7.

$$D = A \cdot B \quad (6)$$

$$E = C \cdot D \quad (7)$$

O algoritmo empregado segue o padrão clássico (Axler, 2015) descrito na Equação 8. A fórmula demonstra a multiplicação de duas matrizes 2×2 , ilustrando o processo de somas e multiplicações. A implementação utilizada pelo *benchmark* está detalhada no Código 12.

$$\begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix} \cdot \begin{bmatrix} b_1 & b_2 \\ b_3 & b_4 \end{bmatrix} = \begin{bmatrix} a_1b_1 + a_2b_3 & a_1b_2 + a_2b_4 \\ a_3b_1 + a_4b_3 & a_3b_2 + a_4b_4 \end{bmatrix} \quad (8)$$

Código 12 – Multiplicação de matrizes.

requer Matrizes *A* de tamanho $n \times m$, *B* de tamanho $m \times p$, *C* de tamanho $n \times p$

```

1: para i = 0 até n, com passo 1 faça
2:   para j = 0 até p, com passo 1 faça
3:     para k = 0 até m, com passo 1 faça
4:        $C[i][j] \leftarrow C[i][j] + A[i][k] \cdot B[k][j]$ 
5:     finaliza para
6:   finaliza para
7: finaliza para
```

Fonte: Adaptado de Pouchet (2011).

O programa executa duas multiplicações de matrizes em sequência. A partir do perfilamento da aplicação, identificou-se que a multiplicação de matrizes corresponde à região computacionalmente mais intensa, consumindo aproximadamente 54% dos ciclos de execução, enquanto a etapa de saída dos dados é responsável por cerca de 44%. Com base nesse perfil, a multiplicação foi selecionada como região alvo para anotação com OpenMP.

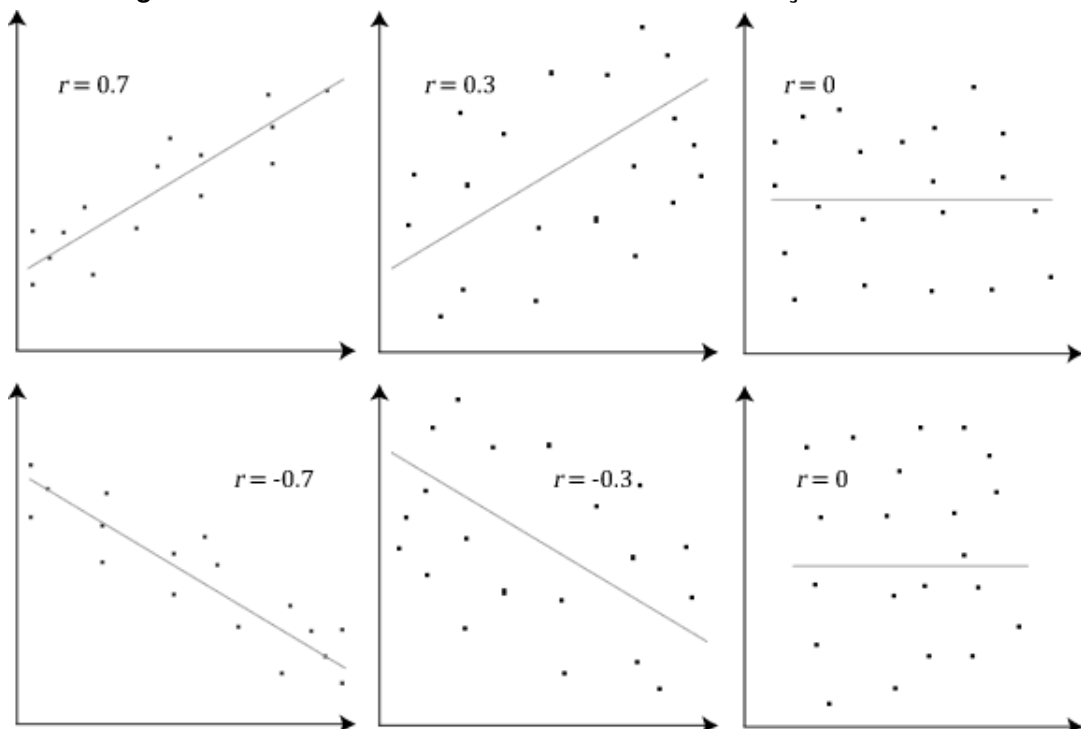
Para reduzir o *overhead* associado à criação de regiões paralelas, utilizou-se uma única região paralela aplicada ao laço da linha 1, que também serviu como escopo para as técnicas de computação aproximada. Na *fastmath*, a aproximação foi aplicada sobre toda a operação; a cláusula *perfo* também foi inserida nessa região; por fim, a técnica *memo* foi empregada na linha 4, memoizando cada interação de k .

Além da paralelização, outra otimização aplicada foi a alocação contígua de memória. Esta técnica também aparece nas outras aplicações que realizam cálculos em matrizes. Em vez de criar um bloco de memória separado para cada linha, o programa aloca um único espaço contínuo. Esta estratégia reduz o número de chamadas de alocação e melhora a localidade espacial.

4.2.2 Correlação

A correlação é uma métrica estatística que quantifica o grau de associação entre duas variáveis (Morettin; Bussab, 2010). A existência de correlação indica que alterações em uma variável tendem a acompanhar alterações na outra. Essa relação é positiva se ambas as variáveis crescem ou decrescem juntas, e negativa se movem em direções opostas. O coeficiente de correlação varia de -1 a 1, nos quais valores próximos a 1 ou -1 apontam para uma forte correlação positiva ou negativa, respectivamente, e valores próximos a 0 denotam associação fraca ou inexistente. A Figura 7 ilustra esses três cenários, permitindo a observação visual do comportamento das variáveis.

Figura 7 – Gráfico do cálculo de coeficiente de correlação de Pearson.



Fonte: Statistics (2019).

O cálculo da correlação é comumente realizado pela fórmula do Coeficiente de Correlação de Pearson, apresentado na Equação 9 (Morettin; Bussab, 2010). Nessa equação, x e y representam os valores das variáveis, e m_x e m_y correspondem às suas respectivas médias. A implementação de correlação do PolyBench (Pouchet, 2011) está descrita nos Códigos 13 e 14.

$$r = \frac{\sum (x - m_x)(y - m_y)}{\sqrt{\sum (x - m_x)^2 \sum (y - m_y)^2}} \quad (9)$$

Código 13 – Construção da matriz de correlação.

```

requer Matrizes  $C$  e  $D$  de tamanho  $n \times m$ 
1: para  $i = 0$  até  $n - 1$ , com passo 1 faça
2:   para  $j = 0$  até  $n - 1$ , com passo 1 faça
3:     se ( $j == i$ ) então
4:        $r \leftarrow 1.0$ 
5:     senão,
6:        $r \leftarrow \text{correlation}(\text{linha}(D, i), \text{linha}(D, j), m)$ 
7:     finaliza se
8:      $C[i][j] \leftarrow r$ 
9:      $C[j][i] \leftarrow r$ 
10:   finaliza para
11: finaliza para

```

Fonte: Adaptado de Pouchet (2011).

Código 14 – Cálculo do coeficiente de correlação de Pearson.

```

requer Arrays  $X$  e  $Y$  de tamanho  $m$ 
1:  $\text{sum}X \leftarrow 0$ 
2:  $\text{sum}Y \leftarrow 0$ 
3:  $\text{sum}XY \leftarrow 0$ 
4:  $\text{sum}X2 \leftarrow 0$ 
5:  $\text{sum}Y2 \leftarrow 0$ 
6: para  $i = 0$  até  $m - 1$ , com passo 1 faça
7:    $\text{sum}X \leftarrow \text{sum}X + X[i]$ 
8:    $\text{sum}Y \leftarrow \text{sum}Y + Y[i]$ 
9:    $\text{sum}XY \leftarrow \text{sum}XY + X[i] \cdot Y[i]$ 
10:   $\text{sum}X2 \leftarrow \text{sum}X2 + X[i] \cdot X[i]$ 
11:   $\text{sum}Y2 \leftarrow \text{sum}Y2 + Y[i] \cdot Y[i]$ 
12: finaliza para
13:  $\text{numerador} \leftarrow m \cdot \text{sum}XY - \text{sum}X \cdot \text{sum}Y$ 
14:  $\text{denominador} \leftarrow \sqrt{(m \cdot \text{sum}X2 - (\text{sum}X \cdot \text{sum}X)) \cdot (m \cdot \text{sum}Y2 - (\text{sum}Y \cdot \text{sum}Y))}$ 
15: retorna  $\text{numerador}/\text{denominador}$ 

```

Fonte: Adaptado de Pouchet (2011).

O resultado do cálculo de correlação compõe uma matriz simétrica, ilustrada na Tabela 2. Os valores na parte inferior da matriz espelham os da parte superior em relação à diagonal principal. Essa diagonal é preenchida com valores 1.0, que representam a autocorrelação perfeita de cada variável. A propriedade de simetria permite otimizações no código, pois passa a ser necessário calcular somente parte das iterações, podendo reutilizar os valores para a outra metade da matriz.

Tabela 2 – Representação da correlação entre as diferentes variáveis.

	a	b	c	d
a	1.0	ab	ac	ad
b	ab	1.0	bc	bd
c	ac	bc	1.0	cd
d	ad	bd	cd	1.0

Fonte: O Autor.

O perfilamento dessa implementação mostrou que os dois códigos apresentados estavam consumindo cerca de 47% dos ciclos. Visando melhor dividir a carga de trabalho as anotações de paralelização foram inseridas no laço de repetição da linha 1 do Código 13. A cláusula `perfo` foi adicionada a este mesmo laço. Já as diretivas `fastmath` e `memo` foram aplicadas no cálculo detalhado no Código 14, ambas aplicadas externamente ao laço de repetição da linha 6.

4.2.3 Deriche

No processamento de imagens, o algoritmo de Deriche aplica uma filtragem recursiva para suavização e detecção de bordas (Deriche, 1987). O método surgiu como uma otimização do detector de Canny. O objetivo é aproximar as propriedades de um filtro Gaussiano por meio de um filtro *Infinite Impulse Response* (IIR).

Diferente de abordagens baseadas em janelas de convolução fixas, o filtro de Deriche armazena e reaproveita os cálculos intermediários de *pixels* vizinhos. Por ser um filtro separável, a operação bidimensional ocorre em etapas unidimensionais independentes. A implementação processa a imagem primeiramente na vertical em ambos os sentidos, e em seguida, processa a matriz na horizontal, também em ambos os sentidos. Essa abordagem proporciona uma suavização eficiente do ruído enquanto preserva as bordas estruturais.

O parâmetro α controla a intensidade da suavização. Ele ajusta o equilíbrio entre a atenuação de ruído e a precisão na localização das bordas. Este parâmetro define os coeficientes a e b , utilizados nas equações de recorrência. As Equações 10 e 11 exemplificam o cálculo unidimensional da varredura horizontal, aplicadas nas direções causal e anti causal, respectivamente. A varredura vertical segue a mesma formulação matemática, alterando apenas os eixos de iteração.

$$y_{ij}^1 = a_1 x_{ij} + a_2 x_{ij-1} + b_1 y_{ij-1}^1 + b_2 y_{ij-2}^1 \quad (10)$$

$$y_{ij}^2 = a_3 x_{ij+1} + a_4 x_{ij+2} + b_1 y_{ij+1}^2 + b_2 y_{ij+2}^2 \quad (11)$$

Código 15 – Algoritmo Deriche de suavização 2D.

```

requer Imagem  $I_{in}$  de tamanho  $h \times w$ 
requer Coeficiente de suavização  $\alpha$ 
1:  $I_{out} \leftarrow$  nova Matriz  $h \times w$  preenchida com zeros
2:  $k \leftarrow \frac{(1-e^{-\alpha})^2}{1+2\alpha e^{-\alpha}-e^{-2\alpha}}$ 
3:  $a_1 \leftarrow k$ 
4:  $a_2 \leftarrow k \cdot e^{-\alpha} \cdot (\alpha - 1)$ 
5:  $a_3 \leftarrow k \cdot e^{-\alpha} \cdot (\alpha + 1)$ 
6:  $a_4 \leftarrow -k \cdot e^{-2\alpha}$ 
7:  $a_5 \leftarrow a_1, a_6 \leftarrow a_2, a_7 \leftarrow a_3, a_8 \leftarrow a_4$ 
8:  $b_1 \leftarrow 2^{-\alpha}$ 
9:  $b_2 \leftarrow -e^{-2\alpha}$ 
10: para  $i = 0$  até  $w - 1$ , com passo 1 faça
11:    $xm_1 \leftarrow 0, ym_1 \leftarrow 0, ym_2 \leftarrow 0$ 
12:   para  $j = 0$  até  $h - 1$ , com passo 1 faça
13:      $in \leftarrow I_{in}[j, i]$ 
14:      $res \leftarrow a_1 \cdot in + a_2 \cdot xm_1 + b_1 \cdot ym_1 + b_2 \cdot ym_2$ 
15:      $I_{out}[j, i] \leftarrow res$ 
16:      $xm_1 \leftarrow in, ym_2 \leftarrow ym_1, ym_1 \leftarrow res$ 
17:   finaliza para
18: finaliza para
19: para  $i = 0$  até  $w - 1$ , com passo 1 faça
20:    $xp_1 \leftarrow 0, xp_2 \leftarrow 0, yp_1 \leftarrow 0, yp_2 \leftarrow 0$ 
21:   para  $j = h - 1$  até 0, com passo  $-1$  faça
22:      $prev \leftarrow I_{out}[j, i]$ 
23:      $res \leftarrow a_3 \cdot xp_1 + a_4 \cdot xp_2 + b_1 \cdot yp_1 + b_2 \cdot yp_2$ 
24:      $I_{out}[j, i] \leftarrow I_{out}[j, i] + res$ 
25:      $xp_2 \leftarrow xp_1, xp_1 \leftarrow res, yp_2 \leftarrow yp_1, yp_1 \leftarrow prev$ 
26:   finaliza para
27: finaliza para
28: para  $j = 0$  até  $h - 1$ , com passo 1 faça
29:    $tm_1 \leftarrow 0, ym_1 \leftarrow 0, ym_2 \leftarrow 0$ 
30:   para  $i = 0$  até  $w - 1$ , com passo 1 faça
31:      $prev \leftarrow I_{out}[j, i]$ 
32:      $res \leftarrow a_5 \cdot prev + a_6 \cdot tm_1 + b_1 \cdot ym_1 + b_2 \cdot ym_2$ 
33:      $I_{out}[j, i] \leftarrow I_{out}[j, i] + res$ 
34:      $tm_1 \leftarrow prev, ym_2 \leftarrow ym_1, ym_1 \leftarrow res$ 
35:   finaliza para
36: finaliza para
37: para  $j = 0$  até  $h - 1$ , com passo 1 faça
38:    $tp_1 \leftarrow 0, tp_2 \leftarrow 0, yp_1 \leftarrow 0, yp_2 \leftarrow 0$ 
39:   para  $i = w - 1$  até 0, com passo  $-1$  faça
40:      $prev \leftarrow I_{out}[j, i]$ 
41:      $res \leftarrow a_7 \cdot tp_1 + a_8 \cdot tp_2 + b_1 \cdot yp_1 + b_2 \cdot yp_2$ 
42:      $I_{out}[j, i] \leftarrow I_{out}[j, i] + res$ 
43:      $tp_2 \leftarrow tp_1, tp_1 \leftarrow prev, yp_2 \leftarrow yp_1, yp_1 \leftarrow res$ 
44:   finaliza para
45: finaliza para
46: retorna  $I_{out}$ 

```

Fonte: Adaptado de Pouchet (2011).

A Figura 8 demonstra o efeito prático do filtro utilizando o coeficiente $\alpha = 1,5$. A Figura 8a apresenta a entrada original em escala de cinza, enquanto a Figura 8b exibe a saída do processamento. O filtro atenua as texturas da imagem, o que facilita o processo computacional de identificar os contornos primários na etapa posterior.

Figura 8 – Aplicação do filtro de Deriche.

(a) Imagem original



(b) Imagem com Deriche aplicado



Fonte: Autoria própria (2026).

O perfilamento desta aplicação mostrou que as chamadas de função relacionadas ao JPEG consomem a maioria dos ciclos de execução, superando inclusive o próprio algoritmo de suavização. Apesar disso, como a implementação avaliada é derivada do trabalho presente no PolyBench (Pouchet, 2011), que não inclui as chamadas de JPEG, optou-se por utilizar o *kernel* de suavização como a região anotada. O Código 15 detalha o *kernel* implementado.

Para paralelizar a aplicação, definiu-se uma única região OpenMP, iniciada na linha 10, englobando as quatro fases de varredura do algoritmo. A cláusula `perfo` foi aplicada aos laços externos das linhas 10, 19, 28 e 37. Já a cláusula `fastmath` foi aplicada externamente, abrangendo as demais regiões de código a partir da linha 10. Por fim, a técnica `memo` foi empregada no corpo dos laços internos de cada varredura, iniciando nas linhas 13, 22, 31 e 40.

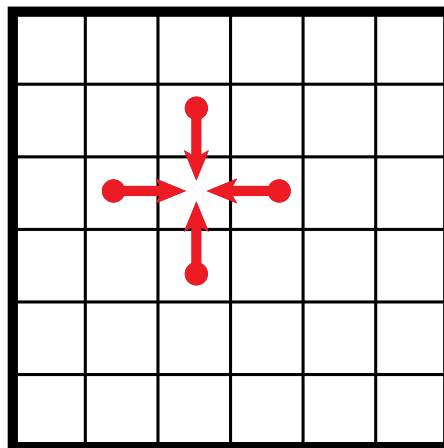
4.2.4 Jacobi 2D

Em análise numérica, o método de Jacobi é um algoritmo iterativo utilizado para resolver sistemas de equações lineares do tipo $Ax = b$ (Szép, 2007). Para que o método funcione adequadamente, o sistema deve atender a algumas condições específicas. A matriz dos coefi-

cientes deve ser quadrada, ou seja, possuir o mesmo número de linhas e colunas. Além disso, o sistema deve ser linear, não podendo conter termos como multiplicação entre variáveis (xy), potências superiores a 1 (x^2 , y^3 , etc.) ou funções não lineares, como seno, cosseno e exponenciais (e^x). Embora não seja uma exigência absoluta, também é recomendável que a matriz apresente dominância diagonal, pois isso contribui para garantir a convergência do método. Na ausência dessa propriedade, o processo iterativo pode apresentar convergência lenta ou até mesmo não convergir

Esse algoritmo foi selecionado do conjunto Polybench (Pouchet, 2011) e adotado neste trabalho por sua natureza iterativa e previsível em termos de comportamento computacional. O processo inicia-se com a atribuição de valores iniciais arbitrários ao vetor x . Em seguida, realiza-se uma iteração para calcular uma nova estimativa da solução, com base exclusivamente nos valores da iteração anterior. Esses novos valores são utilizados como entrada para a próxima iteração. Esse procedimento é repetido até que a diferença entre iterações consecutivas seja inferior a um limiar de tolerância pré-definido, indicando a convergência para uma solução estável.

Figura 9 – Execução do algoritmo de estêncil.



Fonte: Gentryx (2011).

O Jacobi 2D é uma aplicação do método de Jacobi adaptado para problemas em duas dimensões. Sua implementação se baseia em um algoritmo de estêncil, no qual cada ponto de uma malha bidimensional é atualizado com base na média aritmética dos seus quatro vizinhos imediatos (acima, abaixo, esquerda e direita), como mostra a Figura 9. Após um número suficiente de iterações, a malha converge para uma configuração estável que representa uma aproximação da solução contínua do problema. O Código 16 apresenta a implementação utilizada pelo *benchmark*.

O Código 16 apresenta dependência sequencial entre as etapas de atualização, dificultando a paralelização total do algoritmo. Além disso, sua carga computacional é semelhante à observada na aplicação 2MM, sendo dominada por operações sobre matrizes, o que faz com que essa etapa concentre a maioria dos ciclos de execução da aplicação. Para contornar a dependência sequencial, a paralelização foi aplicada no laço mais externo da linha 1, permi-

Código 16 – Método de Jacobi em duas dimensões.

```

requer Matriz  $A$  e  $B$  de tamanho  $n \times n$ 
requer Quantidade de passos do algoritmo  $p$ 
1: para  $t = 0$  até  $p - 1$ , com passo 1 faça
2:   para  $i = 1$  até  $n - 1$ , com passo 1 faça
3:     para  $j = 1$  até  $n - 1$ , com passo 1 faça
4:        $B[i][j] \leftarrow (A[i][j] + A[i][j + 1] + A[i][j - 1] + A[i - 1][j] + A[i + 1][j])/4$ 
5:     finaliza para
6:   finaliza para
7:   para  $i = 1$  até  $n - 1$ , com passo 1 faça
8:     para  $j = 1$  até  $n - 1$ , com passo 1 faça
9:        $A[i][j] \leftarrow (B[i][j] + B[i][j + 1] + B[i][j - 1] + B[i - 1][j] + B[i + 1][j])/4$ 
10:    finaliza para
11:  finaliza para
12: finaliza para

```

Fonte: Adaptado do Pouchet (2011).

tindo que cada etapa de varredura seja executada em paralelo. Já os laços internos da linha 2 e 7, foram anotados com as diretivas `for`, de modo a dividir o trabalho entre as *threads* e sincronizá-las, assegurando que os dados produzidos em uma etapa estejam disponíveis antes da próxima atualização.

A cláusula `perfo` foi aplicada diretamente nas diretivas `for` dos dois laços intermediários. A `fastmath` foi aplicada em todo o bloco de código. E por fim a cláusula `memo` foi aplicada de modo a memoizar o corpo dos laços da linha 3 e 8.

4.2.5 K-means

O algoritmo K-means (Macqueen, 1967) é uma técnica de aprendizado não supervisionado. Seu objetivo é particionar um conjunto de dados em k grupos, garantindo que os elementos de cada grupo sejam similares entre si.

O método define inicialmente k centroides, que atuam como pontos de referência no espaço de características (*features*). Cada ponto do conjunto de dados é associado ao centroide mais próximo utilizando a distância euclidiana. Após a etapa de atribuição, o algoritmo atualiza a posição de cada centroide calculando a média das coordenadas dos pontos pertencentes ao seu respectivo grupo. A Figura 10 demonstra a evolução espacial desse processo. Os triângulos representam os centroides, os círculos indicam os dados e as áreas coloridas demarcam as fronteiras dos agrupamentos formados.

O processo de agrupamento ocorre de forma iterativa. Os dados são reagrupados a cada atualização dos centroides. A execução termina quando o sistema atinge o critério de convergência. Esse critério pode ser um limite máximo de iterações (*iterations*) ou uma variação na posição dos centroides inferior a um limiar específico (*threshold*). O Código 17 (Che *et al.*, 2009) detalha a lógica geral da aplicação, abrangendo a inicialização dos centroides, a atribuição dos pontos, a atualização das coordenadas médias e a verificação de parada.

Código 17 – Algoritmo K-means.

```

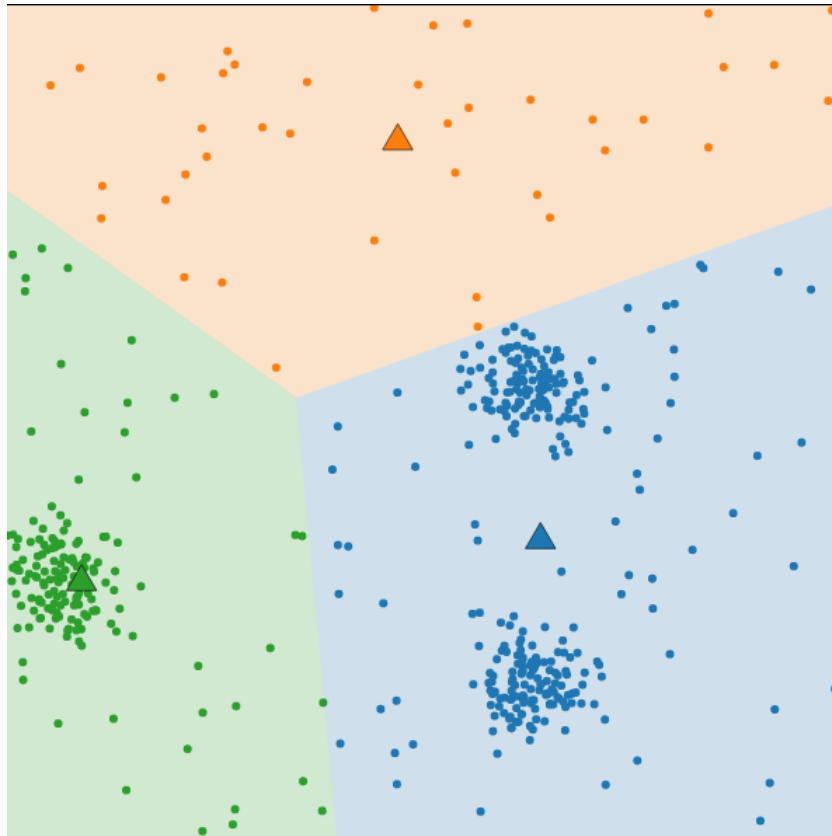
requer Matriz features de dimensão  $numPoints \times numFeatures$ 
requer numClusters, número de clusters que serão formados
requer iterations, limite de iterações do algoritmo
requer threshold, limiar de convergência predefinido
1: Matriz centroids de tamanho  $numClusters \times numFeatures$ , iniciada com 0
2: Matriz newCentroids de tamanho  $numClusters \times numFeatures$ , iniciada com 0
3: Array membership de tamanho  $numPoints$ , iniciado com  $-1$ 
4: Array newCentroidsLen de tamanho  $numClusters$ , iniciado com 0
5: para  $i = 0$  até  $numClusters - 1$  faça
6:    $n \leftarrow$  índice aleatório entre 0 e  $numPoints - 1$ 
7:   para  $j = 0$  até  $numFeatures - 1$  faça
8:      $centroids[i][j] \leftarrow features[n][j]$ 
9:   finaliza para
10: finaliza para
11: para  $iter = 0$  até  $iterations - 1$  faça
12:    $\delta \leftarrow 0$ 
13:   para  $j = 0$  até  $numPoints - 1$  faça
14:      $index \leftarrow \text{find\_nearest\_point}(centroids, numClusters, features, j, numFeatures)$ 
15:     se  $membership[j] \neq index$  então
16:        $\delta \leftarrow \delta + 1$ 
17:     finaliza se
18:      $membership[j] \leftarrow index$ 
19:      $newCentroidsLen[index] \leftarrow newCentroidsLen[index] + 1$ 
20:     para  $k = 0$  até  $numFeatures - 1$  faça
21:        $newCentroids[index][k] \leftarrow newCentroids[index][k] + features[j][k]$ 
22:     finaliza para
23:   finaliza para
24:   para  $j = 0$  até  $numClusters - 1$  faça
25:     se  $newCentroidsLen[j] > 0$  então
26:       para  $k = 0$  até  $numFeatures - 1$  faça
27:          $centroids[j][k] \leftarrow newCentroids[j][k] / newCentroidsLen[j]$ 
28:       finaliza para
29:     finaliza se
30:   finaliza para
31:   Zerar(matriz newCentroids)
32:   Zerar(array newCentroidsLen)
33:   se  $\delta \leq threshold$  então
34:     break
35:   finaliza se
36: finaliza para
37: retorna centroids

```

Fonte: Adaptado de Che *et al.* (2009).

Para determinar a qual agrupamento um ponto pertence, o algoritmo invoca a função de cálculo de distância *find_nearest_point*, detalhada no Código 18. A função compara iterativamente as características de um ponto com as de todos os centroides disponíveis. Visando otimizar o desempenho computacional, a implementação utiliza a distância euclidiana ao quadrado, omitindo a operação de raiz quadrada, uma vez que a relação de grandeza entre as distâncias permanece inalterada. O laço interno acumula a diferença quadrática de cada carac-

Figura 10 – Agrupamentos ao longo das iterações do algoritmo K-means.



Fonte: Incheol (2015).

terística. Sempre que o algoritmo encontra uma distância menor que o menor valor registrado até o momento (*minDist*), ele atualiza o índice do centróide correspondente.

Código 18 – Função *find_nearest_point*.

```

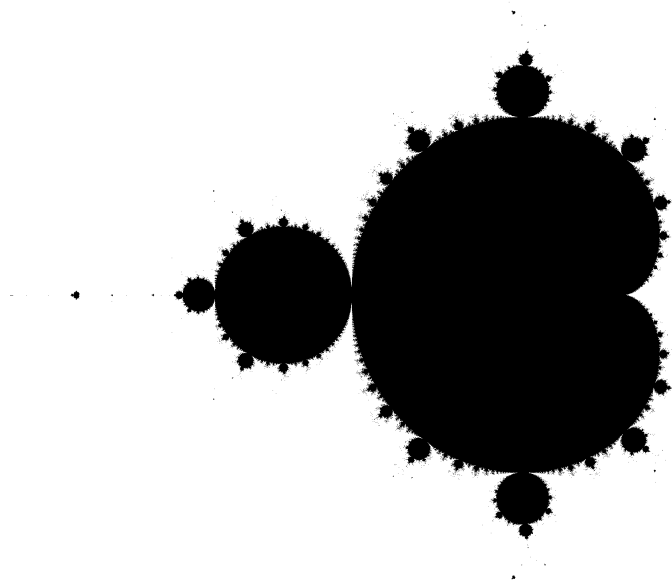
requer Matriz centroids de tamanho  $numClusters \times numFeatures$ 
requer numClusters, número de clusters
requer Matriz features com os dados do problema
requer point, índice do ponto atual a ser avaliado
requer numFeatures, número de características
1: index  $\leftarrow -1$ 
2: minDist  $\leftarrow \infty$ 
3: para i = 0 até numClusters - 1 faça
4:   dist  $\leftarrow 0.0$ 
5:   para j = 0 até numFeatures - 1 faça
6:     diff  $\leftarrow centroids[i][j] - features[point][j]$ 
7:     dist  $\leftarrow dist + (diff \times diff)$ 
8:   finaliza para
9:   se dist < minDist então
10:    index  $\leftarrow i$ 
11:    minDist  $\leftarrow dist$ 
12:   finaliza se
13: finaliza para
14: retorna index

```

As anotações do OpenMP foram aplicadas nos laços das linhas 13 e 24, focando no laço principal que mapeia os pontos de dados (`numPoints`) e no laço que consolida as coordenadas dos agrupamentos (`numClusters`), apresentados no Código 17. A cláusula `perfo` também foi aplicada nesses laços. A cláusula `fastmath` foi aplicada sobre todo Código 18. E `memo` foi aplicada internamente ao laço da linha 13, memoizando os resultados retornados pela função `find_nearest_point`.

4.2.6 Mandelbrot

Figura 11 – Representação do conjunto de Mandelbrot.



Fonte: Autoria própria (2026).

Mandelbrot é um conjunto bidimensional e definido no plano dos números complexos. Sua construção ocorre pela iteração da função apresentada na Equação 12, na qual z e c são números complexos. O valor c permanece constante para cada ponto avaliado, partindo da condição inicial $z_0 = 0$ (Devaney, 1999). Nem todos os valores de c pertencem ao conjunto de Mandelbrot. Um número complexo c faz parte do conjunto se, após aplicar a equação de forma iterativa, os valores de z_n permanecerem limitados. Na prática, considera-se que a sequência diverge quando $|z_n| > 2$, pois a partir desse ponto ela cresce indefinidamente.

$$z_{n+1} = z_n^2 + c \quad (12)$$

Para gerar a representação visual do conjunto, cada *pixel* de uma imagem é mapeado para um número complexo c . As regiões do plano nas quais c pertence ao conjunto recebem então uma representação visual específica. Os valores de c são selecionados em um retângulo no plano complexo. A parte real fica no intervalo $[-2, 1]$ e a parte imaginária no intervalo $[-1,5; 1,5]$. A conversão das coordenadas dos *pixels* para o plano complexo utiliza as Equações 13 e 14. Nessas equações, p_x e p_y representam as coordenadas do *pixel*. Os parâmetros $x_{\min}$ e $x_{\max}$ definem o domínio da parte real, enquanto $y_{\min}$ e $y_{\max}$ definem o domínio da parte imaginária.

$$c_x = x_{\min} + \frac{p_x}{\text{largura da imagem}} \cdot (x_{\max} - x_{\min}) \quad (13)$$

$$c_y = y_{\min} + \frac{p_y}{\text{altura da imagem}} \cdot (y_{\max} - y_{\min}) \quad (14)$$

Os valores c_x e c_y são utilizados como entrada para o cálculo iterativo do conjunto de Mandelbrot. Para cada *pixel* da imagem, a sequência z_n é iterada até que ocorra divergência, ou seja atingido um número máximo de iterações. Caso a sequência permaneça limitada dentro desse limite, o *pixel* correspondente é representado na cor preta, indicando que o ponto pertence ao conjunto de Mandelbrot. A Figura 11 apresenta uma visualização clássica do conjunto gerado. Já o Código 19 (Debian Project, 2018) detalha o cálculo matemático empregado durante as iterações, enquanto o Código 20 descreve a rotina responsável pela geração da imagem.

Código 19 – Cálculo do conjunto de Mandelbrot.

requer Número complexo c

```

1:  $z \leftarrow 0 + 0i$ 
2: para  $i = 0$  até 100, com passo 1 faça
3:    $z \leftarrow z^2 + c$ 
4:   se  $|z| > 2.0$  então
5:     retorna Falso
6:   finaliza se
7: finaliza para
8: retorna Verdadeiro
```

Fonte: Adaptado de Debian Project (2018).

O perfilamento desta aplicação mostra que o Código 19 é responsável por aproximadamente 96% dos ciclos totais de execução. Apesar disso, o algoritmo possui uma instrução `return` dentro do laço principal, o que não é suportado pelo modelo de paralelização do OpenMP. Em razão dessa limitação, a região paralela escolhida foi o laço da linha 5 do Código 20. Essa abordagem evita a criação de uma nova região paralela a cada iteração e permite distribuir entre as *threads* as regiões responsáveis pela maior parte da carga computacional.

Seguindo essa estratégia, a cláusula `perfo` também foi aplicada a nessa mesma região. E as cláusulas `fastmath` e `memo` não apresentam a mesma restrição e, portanto, puderam ser aplicadas sobre todo o bloco do Código 19.

Código 20 – Geração da imagem do conjunto de Mandelbrot.

```

requer Tamanho da imagem base  $n$ 
1:  $imageSize \leftarrow (n/8) \times 8$ 
2:  $pixels \leftarrow$  vetor de tamanho  $(imageSize \times imageSize)/8$ , inicializado com 0
3:  $scaleX \leftarrow (1.0 - (-2.0))/imageSize$ 
4:  $scaleY \leftarrow (1.5 - (-1.5))/imageSize$ 
5: para  $pixel = 0$  até  $tamanho(pixels) - 1$  faça
6:    $byte \leftarrow 0$ 
7:    $byteRow \leftarrow imageSize/8$ 
8:    $pixelColumn \leftarrow pixel \bmod byteRow$ 
9:    $y \leftarrow pixel/byteRow$ 
10:  para  $bit = 0$  até 7 faça
11:     $x \leftarrow pixelColumn \times 8 + bit$ 
12:     $cx \leftarrow -2.0 + x \times scaleX$ 
13:     $cy \leftarrow -1.5 + y \times scaleY$ 
14:    se  $mandelbrot(cx, cy) = \text{Verdadeiro}$  então
15:       $byte \leftarrow byte | (1 \ll (7 - bit))$ 
16:    finaliza se
17:  finaliza para
18:   $pixels[pixel] \leftarrow byte$ 
19: finaliza para

```

Fonte: Adaptado de Debian Project (2018).

4.2.7 PI de Monte Carlo

O método de Monte Carlo engloba algoritmos estatísticos que aplicam amostragem aleatória para solucionar problemas determinísticos ou estimar valores numéricos (Morettin; Bus-sab, 2010). Sua principal característica é a geração de entradas aleatórias em um domínio previamente definido, seguido da aplicação de uma regra ou função para computar a saída, cujo resultado é então agregado para formar uma estimativa.

$$A_{\text{círculo}} = \pi r^2 = \pi \quad (15)$$

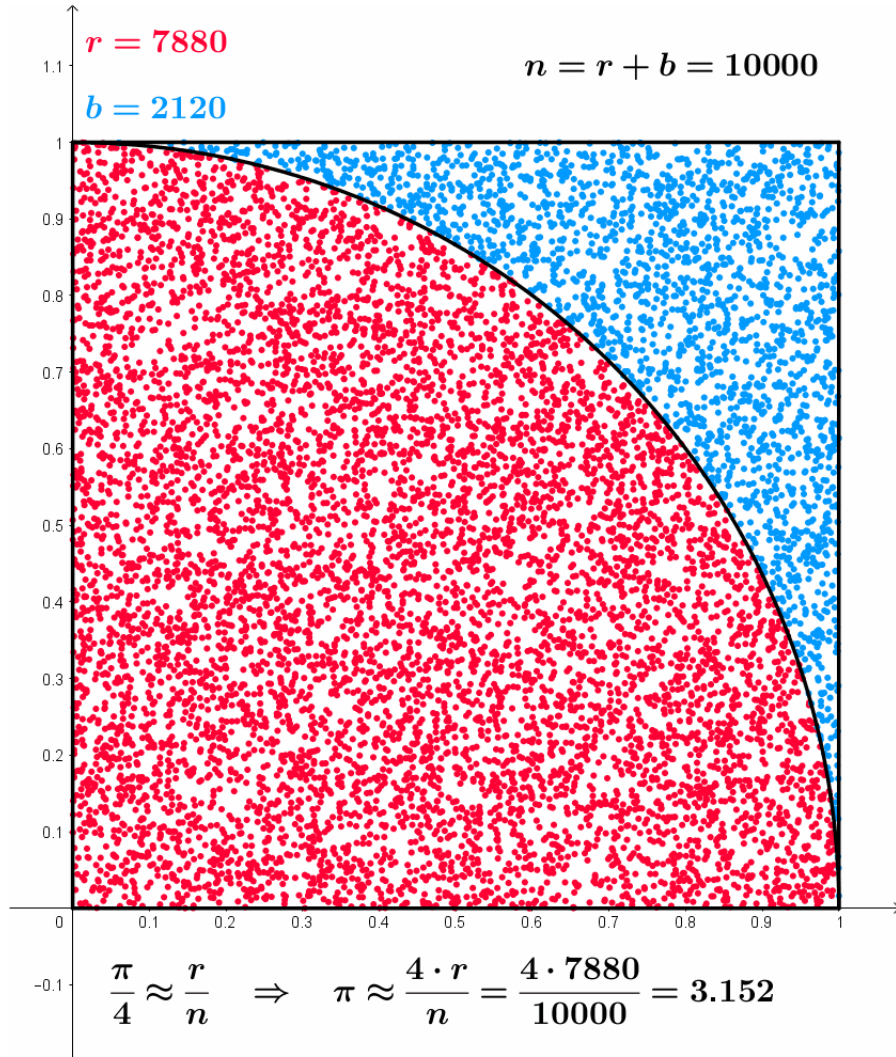
$$A_{\text{quadrado}} = 2 \times 2 = 4 \quad (16)$$

$$\frac{A_{\text{círculo}}}{A_{\text{quadrado}}} = \frac{\pi}{4} \quad (17)$$

$$\pi = 4 \times \frac{A_{\text{círculo}}}{A_{\text{quadrado}}} \quad (18)$$

A aplicação do método para estimar π é intuitiva. Considere um círculo de raio $r = 1$ inscrito em um quadrado de lado 2, como ilustra a Figura 12. As Equações 15 e 16 definem as áreas do círculo e do quadrado. A Equação 17 apresenta a razão entre essas áreas. Isolando o π , obtém-se a Equação 18.

Figura 12 – Método de Monte Carlo



Fonte: Kmhkmh (2023).

O domínio de amostragem corresponde à área do quadrado. O algoritmo gera múltiplos pontos aleatórios dentro desse limite. Para cada coordenada (x,y) , calcula-se então a distância até a origem. O ponto pertence ao círculo se a distância for menor ou igual a 1. A Equação 19 demonstra que a estimativa de π equivale a razão entre a quantidade de pontos contidos no círculo e o total de pontos gerados, multiplicados por 4. A Figura 12 ilustra o processo de amostragem. O Código 21 detalha a implementação adotada no *benchmark*.

$$\pi \approx 4 \times \frac{\text{pontos dentro do círculo}}{\text{total de pontos}} \quad (19)$$

O algoritmo utilizado para a geração de números pseudoaleatórios foi o `xorshift128+` (Vigna, 2017), escolhido principalmente por sua simplicidade e velocidade. A aplicação apresentada no Código 21 possui estrutura reduzida, sendo o laço principal responsável pela maior parte do tempo de execução. Dessa forma, todas as cláusulas de aproximação foram aplicadas nessa região. A paralelização e a cláusula `perfo` foram empregadas diretamente no laço principal, enquanto a cláusula `fastmath` foi aplicada sobre toda a operação

Código 21 – Cálculo de PI pelo método de Monte Carlo.

```

requer Quantidade de iterações  $n$ 
1:  $hit \leftarrow 0$ 
2:  $tid \leftarrow \text{thread\_num}()$ 
3:  $seedState \leftarrow \{tid, tid + 1\}$ 
4: para  $i = 0$  até  $n$ , com passo 1 faça
5:    $x \leftarrow \text{random}(seedState)$ 
6:    $y \leftarrow \text{random}(seedState)$ 
7:   se  $(x \times x + y \times y) \leq 1$  então
8:      $hit \leftarrow hit + 1$ 
9:   finaliza se
10: finaliza para
11: retorna  $(4.0 \times hit)/n$ 

```

Fonte: Autoria própria (2026).

realizada nessa região. Por fim, a técnica `memo` foi inserida a partir da linha 5, para memoizar os valores da variável `hit`.

4.3 Resumo do conjunto de aplicações

Tabela 3 – Resumo das aplicações utilizadas nos experimentos e seus respectivos tamanhos de entrada.

Aplicação	Fonte	Domínio	Entrada	Métrica
2MM	PolyBench	Álgebra Linear	Matrizes 2048×2048	MAPE
Correlação	PolyBench	Probabilidade e Estatística	Matriz 8192×256 (observações $\times$ variáveis)	MAPE
Deriche	PolyBench	Processamento de Imagem	Imagem (8a)	SSIM
Jacobi 2D	PolyBench	Solução Numérica	Matriz 2048×2048 , 100 passos	MAPE
K-Means	Rodinia	Aprendizado de Máquina	1.313.220 pontos, 10 <i>clusters</i> , 100 iterações, limiar 5	MCR
Mandelbrot	Debian Benchmarks Game	Visualização Computacional	Imagem 2048×2048	MAPE
PI de Monte Carlo	Debian Benchmarks Game	Probabilidade e Estatística	10^9 iterações	MAPE

Fonte: Autoria própria (2026).

As aplicações escolhidas apresentam diferentes padrões computacionais, possuem suporte à paralelização e incluem etapas de execução tolerantes a aproximações. A Tabela 3 resume os *benchmarks* utilizados, detalhando a fonte, o domínio do problema e a respectiva métrica de qualidade. Cada programa avaliado serviu para avaliar o impacto das técnicas de aproximação no desempenho e na precisão dos cálculos.

O tamanho das entradas de cada aplicação foi definido com base nos padrões já definidos nos seus *benchmarks* de origem. As métricas de qualidade variam conforme a natureza da aplicação. Para programas com saídas numéricas contínuas, como 2MM, Correlação, Jacobi 2D e PI de Monte Carlo, adotou-se o MAPE. Essa métrica calcula a média do erro absoluto de forma percentual, dividindo a diferença entre a saída aproximada e a exata pelo próprio valor exato. Diferente de métricas simétricas, o MAPE quantifica o desvio tendo o valor correto como referência absoluta. Isso facilita a interpretação da perda de precisão causada pela aproximação computacional.

No K-Means, utilizou-se o MCR, uma métrica específica e apropriada para calcular a taxa de classificações incorretas na atribuição dos pontos aos *clusters*. No caso do Deriche e do Mandelbrot, cujo processamento gera uma imagem, optou-se pelo SSIM, que quantifica matematicamente a similaridade estrutural e visual entre a imagem gerada e a referência correta.

5 RESULTADOS

Este capítulo apresenta os resultados obtidos a partir da aplicação das técnicas de CA ao conjunto de *benchmarks* selecionados. A análise considera tanto o desempenho quanto a qualidade das saídas produzidas. Os gráficos comparam execuções aproximadas e não aproximadas em diferentes configurações de *threads*, destacando os impactos das aproximações no erro das saídas e no tempo de execução em relação ao *baseline*.

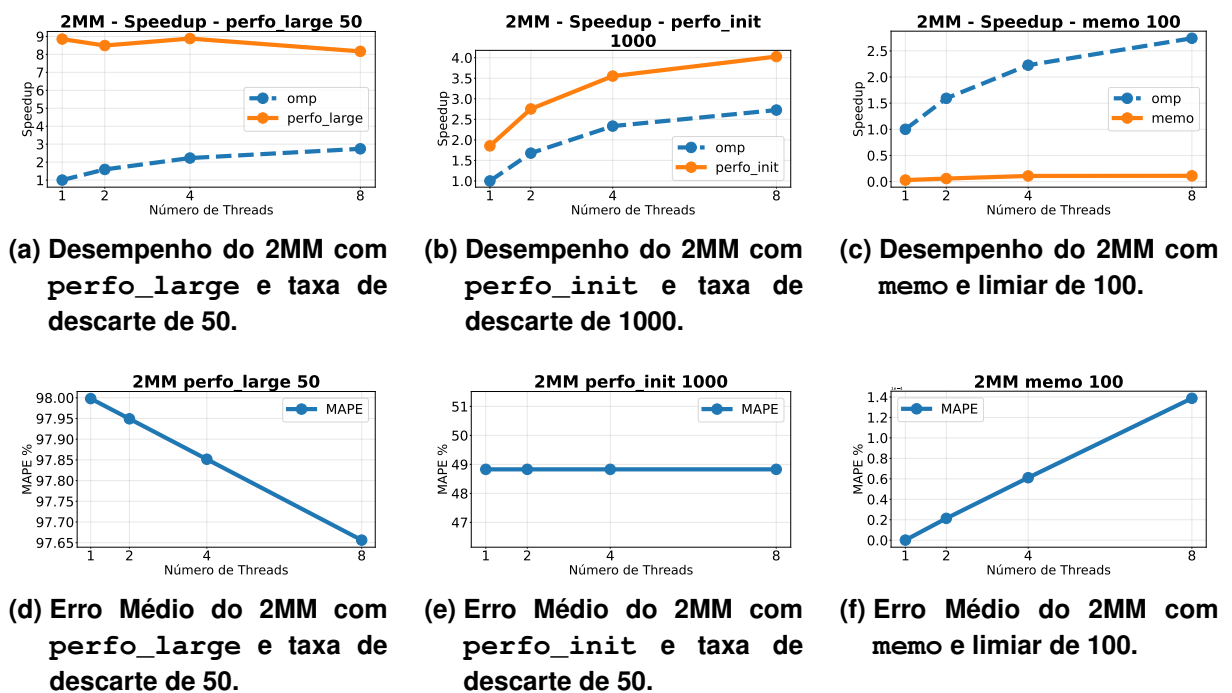
5.1 Análise por Aplicação

Esta seção apresenta a análise individual de cada aplicação do conjunto de *benchmarks*. Para cada programa, são discutidos os efeitos das técnicas de aproximação sobre o tempo de execução e o erro dos resultados, destacando limitações observadas e possíveis características que influenciam a sensibilidade à aproximação.

5.1.1 2MM

Em relação ao erro e ao desempenho, conforme ilustrado na Figura 13, a técnica *perfo_large* apresentou os maiores resultados; em termos de qualidade, essa técnica apresentou um erro médio que variou de 90.6% a 97.9%, dependendo da taxa de perfuração e do número de *threads*. Em contrapartida, essa mesma redução no número de iterações foi responsável pe-

Figura 13 – Técnicas baseadas em perfuração de laço entregaram os melhores ganhos de tempo de execução, porém sacrificaram a precisão dos resultados.



Fonte: Autoria própria (2026).

los maiores ganhos de desempenho observados. Considerando a execução sem aproximação com uma única *thread*, cujo tempo médio foi de aproximadamente 11,35 segundos, a versão com taxa de aproximação 50 reduziu o tempo médio para cerca de 1,21 segundos, o que corresponde a um ganho próximo de $9,4\times$. Mesmo em configurações menos agressivas, como taxa 20 ou 30, os tempos médios permaneceram abaixo de 1,49 segundos, mantendo ganhos superiores a $7\times$.

As técnicas *perfo_init* e *perfo_fini* apresentaram comportamento semelhante entre si, tanto em termos de erro quanto de desempenho. O erro médio aumentou de forma consistente com o crescimento da taxa de aproximação. Para uma taxa de 10, o erro médio foi de aproximadamente 49%. Com o aumento no nível de descarte, o erro também cresceu, atingindo cerca de 4,88% para taxa 100, 24,41% para taxa 500 e aproximadamente 48,83% para taxa 1000. Em relação ao desempenho, a execução com uma única *thread* na versão com a taxa de descarte de 500 reduziu o tempo médio de execução de aproximadamente 11,35 segundos para cerca de 7,96 segundos, enquanto a versão com taxa 1000 reduziu esse tempo para aproximadamente 5,83 segundos. Esses valores correspondem a ganhos superiores a $3\times$ quando comparados à *baseline* com o mesmo número de *threads*. Para 8 *threads*, por exemplo, a execução sem aproximação apresentou tempo médio de aproximadamente 3,89 segundos, enquanto a versão com taxa 1000 reduziu esse valor para cerca de 2,68 segundos.

A técnica de memoização apresentou erro praticamente desprezível em todas as configurações avaliadas. Esse comportamento está diretamente relacionado ao mecanismo da cláusula responsável por evitar *overhead*, no qual o sistema para de realizar memoização na região quando o custo adicional passa a superar os benefícios esperados. Mesmo assim houve uma perda de desempenho com essa técnica em todas as configurações testadas. Esse comportamento é justificado pelo custo adicional associado às estruturas de memoização, que foram ativadas para cada execução do laço mais interno, gerando um *overhead* significativo na aplicação.

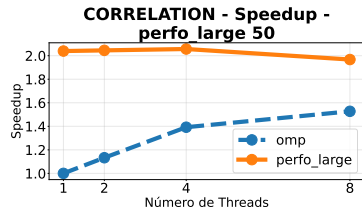
Por fim, a técnica *fastmath* não introduziu alterações significativas no erro dos resultados, indicando que as otimizações de ponto flutuante habilitadas pelo compilador não modificaram de forma relevante a precisão numérica da aplicação. Em termos de desempenho, essa técnica também não apresentou ganhos em nenhuma das configurações avaliadas.

Uma análise de perfilamento com a ferramenta *valgrind*, mostrou que a execução do *perfo_large* reduziu a região de multiplicação de matrizes para apenas 3,8% dos ciclos, tornando a função de saída o gargalo da aplicação (consumindo 92,94% dos ciclos). Também foi possível observar que, embora tenha ocorrido uma redução no número de instruções executadas dentro dos laços, houve um aumento relativo no custo das barreiras de sincronização, uma vez que as *threads* terminam seu trabalho mais rapidamente e passam a aguardar nesses pontos de sincronização. De forma geral, os resultados mostram que as técnicas baseadas em perfuração de laço foram as mais eficazes para reduzir o tempo de execução da aplicação, mas esse ganho ocorre às custas de uma baixa acurácia nos resultados.

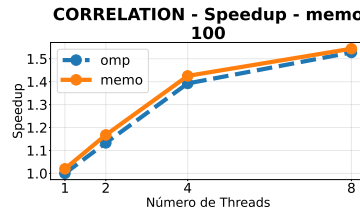
5.1.2 Correlação

Figura 14 – `perfo_large` foi a técnica que apresentou o maior ganho de desempenho da aplicação, enquanto as demais técnicas falharam em trazer ganhos.

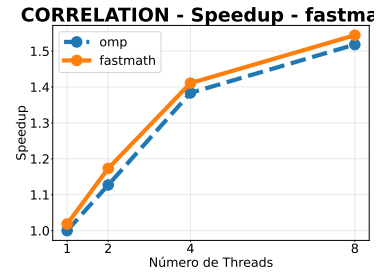
(a) Desempenho do Correlation com `perfo_large` e taxa de descarte de 50.



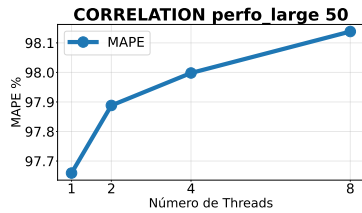
(b) Desempenho do Correlation com `memo` e limiar de 100.



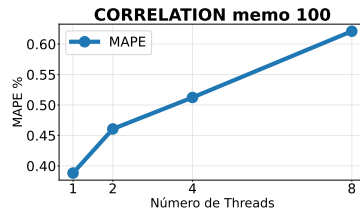
(c) Desempenho do Correlation com `fastmath`.



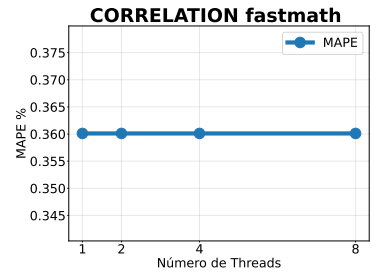
(d) Erro Médio do Correlation com `perfo_large` e taxa de descarte de 50.



(e) Erro Médio do Correlation com `memo` e limiar de 100.



(f) Erro Médio do Correlation com `fastmath`.



Fonte: Autoria própria (2026).

A técnica `perfo_large` apresentou o maior impacto tanto em desempenho quanto em qualidade, como evidenciado na Figura 14. Observou-se um ganho de desempenho consistente em todas as configurações, com tempos médios de execução próximos de 0,35 segundos, enquanto a execução sem aproximação com uma única *thread* apresentou tempo médio de aproximadamente 0,74 segundos. Esse comportamento representa um ganho de desempenho próximo de 2×. Esse resultado está diretamente associado à redução significativa do número de iterações executadas no laço mais interno, o que diminuiu o custo computacional da aplicação.

Essa redução veio com o custo de uma degradação acima de 90% na qualidade dos resultados. O erro médio permaneceu elevado em todas as configurações, variando de aproximadamente 90,96% no menor nível de aproximação até cerca de 98,14% no maior nível. Esse resultado sugere que o cálculo de correlação depende fortemente da avaliação completa das amostras que a omissão de parte dos dados compromete a acurácia do algoritmo.

Já as técnicas `perfo_init` e `perfo_fini` apresentaram um comportamento distinto do `perfo_large`. Mesmo nos níveis mais agressivos de aproximação, o erro médio permaneceu abaixo de 1%, com valores próximos de 0,375%. Em níveis menores de aproximação, o erro chegou a aproximadamente 0,36%, indicando que a remoção de um subconjunto

dos dados não compromete de forma significativa a precisão do cálculo. Essas técnicas também não apresentaram ganhos significativos de desempenho, apresentando valores muito próximos de uma execução sem aproximação, indicando que a redução no número total de iterações não foi suficiente para alterar de forma significativa o custo computacional dominante da aplicação.

A `memo` apresentou um comportamento equilibrado entre desempenho e qualidade. O erro médio permaneceu baixo em todas as configurações, variando de aproximadamente 0,39% até cerca de 0,79% no pior caso, mesmo com o aumento do número de *threads*. Esse resultado indica que a reutilização de valores previamente computados preservou a estabilidade numérica do algoritmo. Em termos de desempenho, o ganho não foi muito expressivo, o tempo médio de execução com 8 *threads* foi de aproximadamente 0,466 segundos, enquanto a execução sem aproximação apresentou tempo médio de cerca de 0,474 segundos.

Já a `fastmath` apresentou resultados semelhantes aos observados com `memo`. O erro médio permaneceu estável em aproximadamente 0,36% em todas as configurações de paralelismo, indicando que as otimizações de ponto flutuante introduzidas pelo compilador não comprometeram de forma significativa a precisão do cálculo. E em termos de desempenho, observou-se um resultado similar ao do `memo`, ou seja, não foram valores muito expressivos.

A análise de perfilamento indicou que o laço mais interno é responsável por cerca de 22% dos ciclos totais, enquanto a região principal da aplicação consumiu aproximadamente 47,74%. Com a aplicação da técnica `perfo_large`, o consumo de ciclos do laço foi reduzido para aproximadamente 1,68%, tornando a leitura do arquivo o gargalo principal da aplicação. Essa redução também foi observada com o `memo`, mas com essa técnica o ganho foi parcialmente compensado pelo custo adicional introduzido pela *runtime*.

De forma geral, os resultados dessa aplicação indicam que técnicas que preservam a maioria do espaço de iteração, como `memo`, `fastmath`, `perfo_init` e `perfo_fini`, mantêm a qualidade dos resultados com impacto reduzido no desempenho. Por outro lado, técnicas que alteram diretamente a frequência de execução do laço principal, como `perfo_large`, produziram ganhos mais expressivos de desempenho, ao custo de uma degradação significativa na precisão.

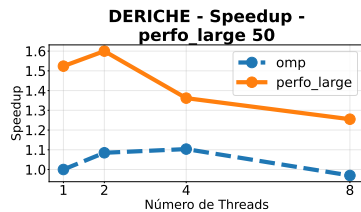
5.1.3 Deriche

A avaliação da aplicação Deriche revela como diferentes estratégias de aproximação impactam o tempo de execução e a qualidade visual da imagem gerada, resultados estes consolidados na Figura 15. Para estabelecer uma base de comparação, a execução sem aproximação obteve tempos médios de 0,064 segundos com uma *thread* e 0,058 segundos com quatro *threads*.

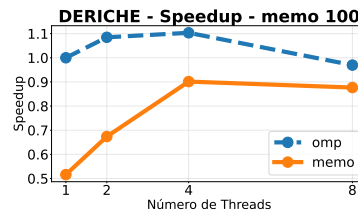
A técnica `perfo_large` apresentou ganhos de desempenho consistentes, alcançando acelerações de até $1,6\times$ em relação à execução original. O tempo de execução variou entre 0,040 e 0,049 segundos nas execuções aproximadas que utilizaram de 1 a 4 *threads*, o que apresenta um leve ganho se comparado com o tempo médio da execução sem aproximação

Figura 15 – Técnicas de perfuração proporcionaram ganhos significativos de desempenho sem comprometer de forma expressiva a saída. Em contrapartida, a memoização apresentou um efeito duplamente negativo, pois não gerou aceleração e ainda reduziu a acurácia dos resultados.

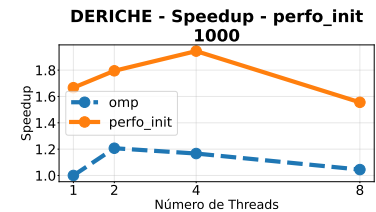
(a) Desempenho do Deriche com `perfo_large` e taxa de descarte de 50.



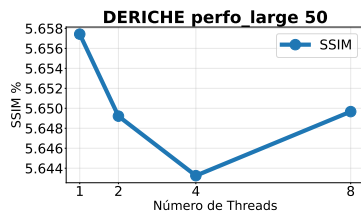
(b) Desempenho do Deriche com `memo` e limiar de 100.



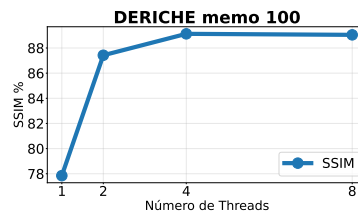
(c) Desempenho do Deriche com `fastmath`.



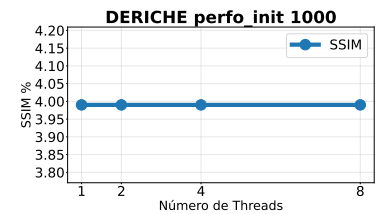
(d) Erro Médio do Deriche com `perfo_large` e taxa de descarte de 50.



(e) Erro Médio do Deriche com `memo` e limiar de 100.



(f) Erro Médio do Deriche com `fastmath`.



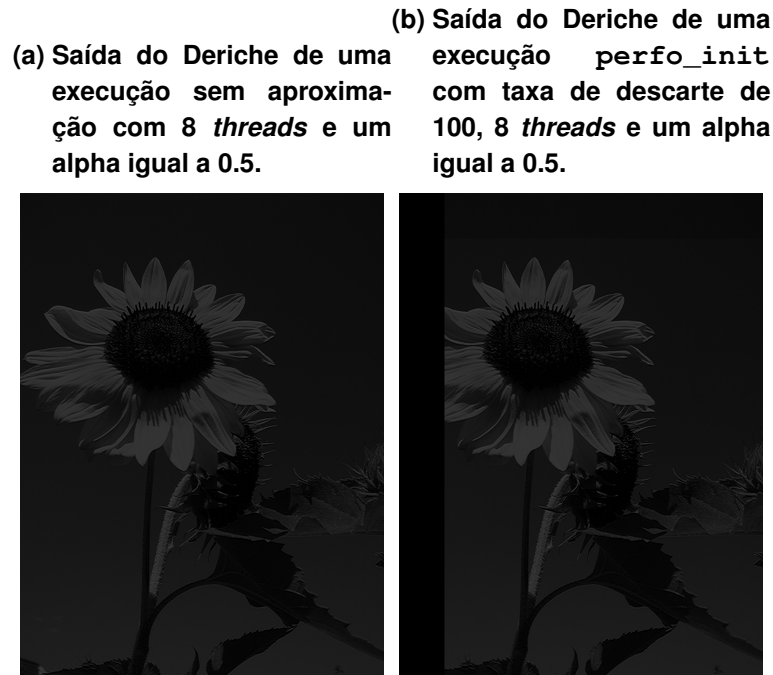
Fonte: Autoria própria (2026).

que foi de 0,064 segundos com 1 *thread* e 0,058 segundos com 4 *threads*. Contudo, a similaridade da imagem gerada em relação à original sofreu uma degradação, registrando valores próximos a 12% de similaridade com a taxa de aproximação configurada em 10, caindo para aproximadamente 5,6% com a taxa em 50.

As estratégias `perfo_init` e `perfo_fini` exibiram comportamentos semelhantes entre si. A qualidade dos dados caiu de forma proporcional ao tamanho do `drop`. Taxas menores, como o valor 10, mantiveram a similaridade em torno de 99%; valores intermediários, como 100, reduziram a qualidade para cerca de 87,6%; e, por fim, taxas agressivas, com saltos de 500 e 1000, produziram erros de 39,7% e 4%, respectivamente. Em contrapartida, as configurações de 500 e 1000 proporcionaram reduções substanciais no tempo de execução, atingindo marcas de 0,036 segundos para `perfo_init` e 0,037 segundos para `perfo_fini` com o uso de 4 *threads*.

As saídas visuais podem ser comparadas na Figura 16. A cláusula `fastmath` não alterou o aspecto visual de forma prática, mantendo uma taxa de similaridade estável em 99,99%. Apesar da alta precisão preservada, a adoção dessa técnica gerou uma leve perda de desempenho, o que pode ser explicado pelo fato de que a região precisou ser extraída para que as otimizações fossem aplicadas, gerando um *overhead* de chamadas. Os tempos de execução ficaram na casa de 0,065 segundos para uma *thread*, um valor superior à base de comparação comum. A estratégia `memo`, de maneira similar, não obteve vantagens de tempo de execução

Figura 16 – A perfuração ao ser aplicada no algoritmo omite parte da imagem de saída..



Fonte: Autoria própria (2026).

se comparado com a execução sem aproximação, embora tenha alcançado um padrão de qualidade aceitável com valores próximos a 80% de similaridade na maior parte dos testes.

O perfilamento da aplicação mostrou que o maior número de ciclos da aplicação é gasto no processo de escrita do arquivo no formato JPEG. Como as técnicas de aproximação geram imagens com um nível significativamente menor de detalhes, as etapas de compressão e quantização nativas do codificador JPEG tornam-se mais rápidas. Isso reduz o tempo total da aplicação de forma indireta, o que explica os ganhos nas estratégias de perfuração.

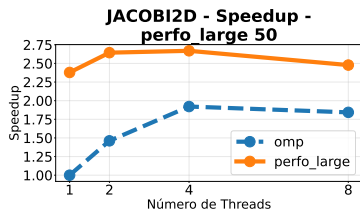
5.1.4 Jacobi 2D

O Jacobi 2D foi a aplicação que apresentou um baixo nível de erro em todos os tipos de aproximação, se comparado com as demais aplicações, o que pode ser verificado na Figura 17. A técnica `perfo_large` registrou um erro médio em torno de 0,03%, um valor baixo em comparação com a execução original. As estratégias `perfo_init` e `perfo_fini` apresentaram perdas de precisão ainda menores, mantendo-se consistentemente abaixo de 0,01%. A diretiva `fastmath` manteve a integridade total dos cálculos, não introduzindo qualquer desvio nos valores. E por fim a técnica `memo` também manteve um erro médio baixo na grande maioria dos cenários, com a única exceção na taxa de aproximação igual a 100 e 8 *threads*, na qual o erro foi de 17,5%.

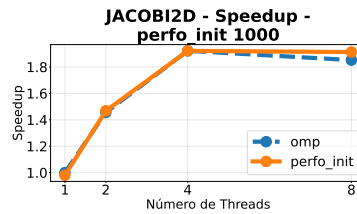
Já na questão de desempenho, a técnica `perfo_large` foi a única que apresentou ganhos significativos em todas as configurações, chegando a ganhos de até $2,75\times$ em relação à execução original, e mantendo uma aceleração superior a $2\times$ em todas as configurações

Figura 17 – Enquanto a `perfo_large` gerou ganhos de desempenho, demais técnicas não foram suficientes para ultrapassar a execução sem aproximação.

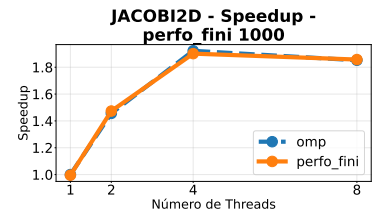
(a) Desempenho do Jacobi 2D com `perfo_large` e taxa de descarte de 50.



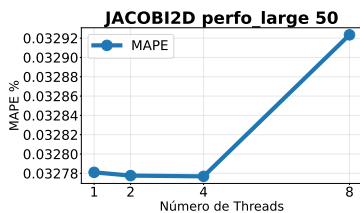
(b) Desempenho do Jacobi 2D com `perfo_init` e taxa de descarte de 1000.



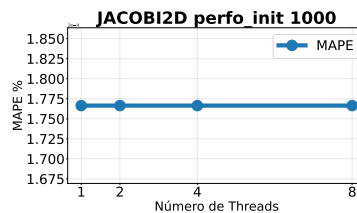
(c) Desempenho do Jacobi 2D com `perfo_fini` e taxa de descarte de 1000.



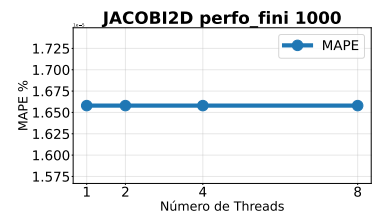
(d) Erro Médio do Jacobi 2D com `perfo_large` e taxa de descarte de 50.



(e) Erro Médio do Jacobi 2D com `perfo_init` e taxa de descarte de 1000.



(f) Erro Médio do Jacobi 2D com `perfo_fini` e taxa de descarte de 1000.



Fonte: Autoria própria (2026).

avaliadas. A execução sem aproximação com uma *thread* leva cerca de 5,94 segundos para ser concluída, enquanto a aplicação com a `perfo_large` e uma taxa de 50 e a mesma quantidade de *threads* leva somente 2,49 segundos.

A técnica `memo` gerou perda de desempenho em todas suas versões, chegando até a ultrapassar 22 segundos no cenário com apenas uma *thread*. Esse atraso é justificado pelo alto *overhead* inserido a cada iteração para o gerenciamento das estruturas de dados da memoização, que assim como no 2MM foi realizado a cada iteração do laço mais interno. A cláusula `fastmath` também não produziu melhorias no tempo de execução desta aplicação específica.

Por fim, a análise da aplicação por meio de ferramentas de perfilamento indicou que essa aplicação segue o mesmo modelo do 2MM, onde as técnicas de perfuração reduziram a quantidade de ciclos da execução da função, porém provocaram um leve aumento no tempo gasto com barreiras de sincronização. Contudo, no cenário específico da `perfo_large`, a redução drástica no volume total de instruções processadas compensou o custo extra de controle das *threads*.

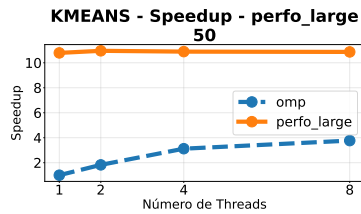
5.1.5 K-Means

O *K-Means* apresenta como principal gargalo de desempenho a etapa de busca pelos pontos mais próximos aos centroides. O comportamento do algoritmo com as aproximações pode ser observado na Figura 18. O perfilamento da execução sem aproximação demonstra que essa fase consome aproximadamente 59 bilhões de ciclos de processamento, o que representa

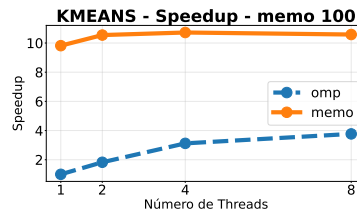
Figura 18 – Aplicação apresentou ganhos de desempenho com o uso das diferentes técnicas de aproximação, porém não sem gerar uma degradação na qualidade dos resultados.

(a) Desempenho do K-Means

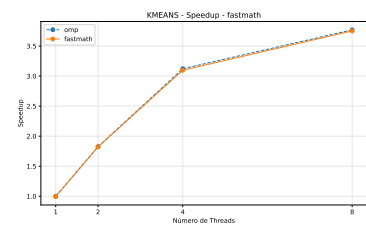
com `perfo_large` e taxa de descarte de 50.



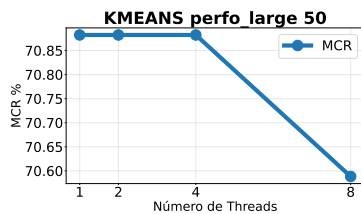
(b) Desempenho do K-Means
com `memo` e limiar de 100.



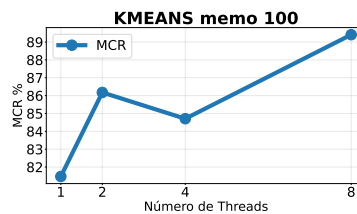
(c) Desempenho do K-Means
com `fastmath`.



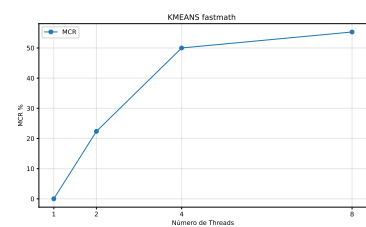
(d) Erro Médio do K-Means
com `perfo_large` e taxa de descarte de 50.



(e) Erro Médio do K-Means
com `perfo_init` e taxa de descarte de 100.



(f) Erro Médio do K-Means
com `fastmath`.



Fonte: Autoria própria (2026).

39,88% do tempo total da aplicação. A execução da aplicação com 1 *thread* executa em uma média de 14,83 segundos, enquanto a versão com 8 *threads* conclui o processamento em 3,93 segundos.

A aplicação da técnica `perfo_large` reduziu a proporção de ciclos na região crítica de 39,88% para apenas 0,43%. Essa diminuição drástica de processamento gerou ganhos de desempenho superiores a 10× em relação à execução sem aproximação. Os tempos de execução mantiveram valores entre 1,35 e 1,49 segundos em todas as configurações de *threads* e taxas de aproximação. Contudo, a métrica de erro médio ficou estagnada em torno de 70% em todos os cenários.

As estratégias `perfo_init` e `perfo_fini` apresentaram comportamentos similares. A quantidade de ciclos gastos na região crítica caiu para 14,90% dos ciclos com essas técnicas. O tempo de execução variou de 2,18 segundos com uma *thread* até 1,50 segundos com 8 *threads*. Apesar de garantirem um ganho de velocidade próximo a 10×, a perda na acurácia foi similar ao `perfo_large`, mantendo o erro na faixa de 70%.

A técnica `memo` reduziu os ciclos da região crítica para 0,33%. No entanto, o custo das suas estruturas internas introduziu alguns ciclos extras durante a execução. Nas taxas de aproximação de 10, 25 e 50, a técnica prejudicou o desempenho geral da aplicação. O tempo de execução com uma *thread* subiu para cerca de 15,6 segundos, e com 8 *threads* ficou em torno de 4,19 segundos. O ganho de desempenho expressivo ocorreu apenas na taxa máxima de 100, atingindo 1,39 segundos com 8 *threads*.

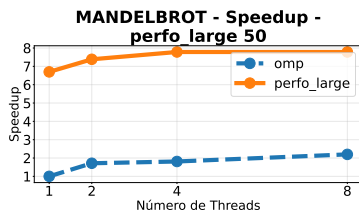
A qualidade dos dados gerados pela cláusula `memo` sofreu degradação diretamente proporcional ao aumento do número de *threads* e da taxa de aproximação. Com uma *thread* e taxas de até 50, a execução não apresentou erro. Porém, ao elevar o paralelismo para 8 *threads* na mesma taxa de 50, o erro saltou para 55,29%. Na taxa extrema de 100, o erro alcançou 81,47% na execução sequencial e chegou a 89,41% com 8 *threads*. A técnica `fastmath`, que habilita otimizações agressivas de ponto flutuante via compilador, reproduziu o mesmo padrão de perda de acurácia da memoização, mas falhou em entregar qualquer ganho de desempenho prático em relação à execução original.

Desse modo, temos que o algoritmo *K-Means* conseguiu obter ganhos de desempenho com as diferentes técnicas de aproximação empregadas, porém ele se mostrou sensível a elas em relação à acurácia de saída, chegando a níveis de degradação superiores a 70%.

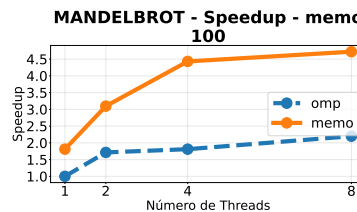
5.1.6 Mandelbrot

Figura 19 – Aplicação apresentou bons resultados tanto no desempenho quanto na acurácia nas técnicas `perfo_large` e `memo`.

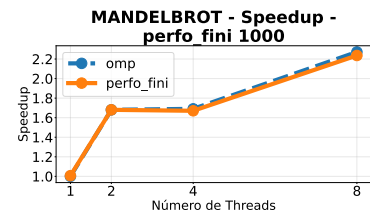
(a) Desempenho do Mandelbrot com `perfo_large` e taxa de descarte de 50.



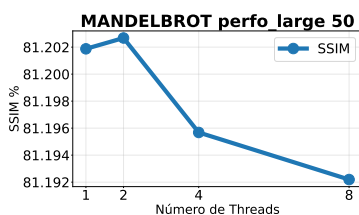
(b) Desempenho do Mandelbrot com `memo` e limiar de 100.



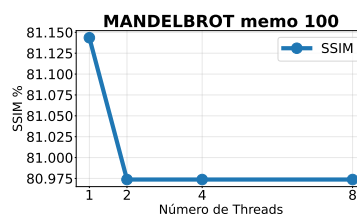
(c) Desempenho do Mandelbrot com `perfo_fini` e taxa de descarte de 1000.



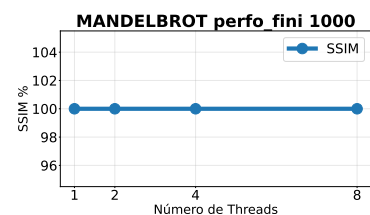
(d) Erro Médio do Mandelbrot com `perfo_large` e taxa de descarte de 50.



(e) Erro Médio do Mandelbrot com `perfo_init` e limiar de 100.



(f) Erro Médio do Mandelbrot com `perfo_fini` e taxa de descarte de 1000.



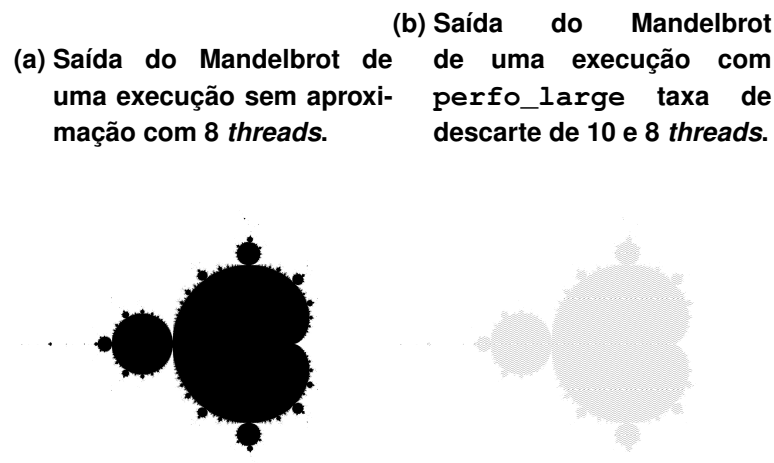
Fonte: Autoria própria (2026).

As cláusulas `perfo_init`, `perfo_fini` e `fastmath` não geraram alterações nas imagens em comparação com a execução original, como demonstrado na Figura 19. Por outro lado, a técnica `perfo_large` manteve a similaridade visual em aproximadamente 80%, apresentando apenas leves variações. A técnica `memo` também alcançou valores de similaridade muito próximos aos da versão original na maioria dos cenários. Essa similaridade variou de 81% em execuções seriais até mais de 99% em configurações com múltiplas *threads* e taxas

de descarte menores, com a única exceção na taxa de aproximação igual a 100, na qual a similaridade estabilizou em torno de 80% independentemente da quantidade de *threads* utilizadas.

Já na questão de desempenho, a estratégia `perfo_large` obteve os melhores valores, apresentando ganhos superiores a 6× em todas as configurações testadas, chegando a acelerações de até 8× em relação à execução original. A execução sem aproximação com uma *thread* leva cerca de 0,288 segundos para ser concluída, enquanto a aplicação com a `perfo_large` reduz esse tempo para o intervalo de 0,036 a 0,055 segundos.

Figura 20 – Mesmo com a aplicação da técnica de aproximação o formato do conjunto foi gerado corretamente.



Fonte: Autoria própria (2026).

Exemplos dessas saídas visuais estão expostos na Figura 20. A técnica `memo` obteve ganhos exclusivamente na configuração com taxa 100, alcançando acelerações de até 5× em relação à execução original. Nessa configuração específica, o tempo reduziu de 0,32 segundos na *baseline* serial para 0,159 segundos, atingindo 0,061 segundos quando escalado para 8 *threads*. Contudo, nas demais taxas de aproximação (10, 25 e 50), a técnica sofreu perda de desempenho. As abordagens `perfo_init`, `perfo_fini` e `fastmath` também não produziram melhorias no tempo de execução desta aplicação.

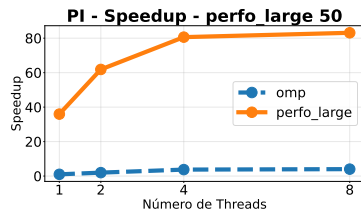
5.1.7 PI de Monte Carlo

Ambas as técnicas `perfo_init` e `perfo_fini` apresentaram uma variação de erro inferior a 0,01% e não obtiveram ganho de desempenho, o que é corroborado pelos dados da Figura 21. Isso é explicado, pois a quantidade de iterações que foram saltadas não foi suficiente para superar o custo de criação e gerenciamento do laço paralelo. A técnica `fastmath` acompanhou esse mesmo comportamento, apresentando erro semelhante e nenhuma melhora no tempo de execução.

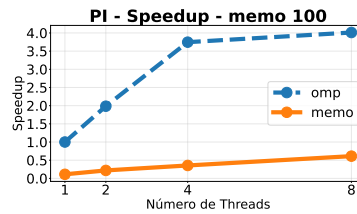
Por outro lado, a técnica `perfo_large` com a configuração de descarte de 50 e alocação de 4 *threads* alcançou um tempo de 0,033 segundos, o que contrasta diretamente com os

Figura 21 – A memoização foi uma técnica que não só diminuiu o desempenho da aplicação como também gerou uma saída completamente fora do esperado.

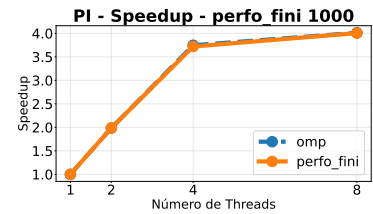
(a) Desempenho de PI com `perfo_large` e taxa de descarte de 50.



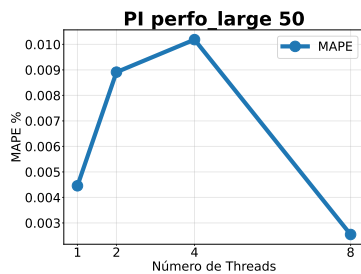
(b) Desempenho de PI com `memo` e limiar de 100.



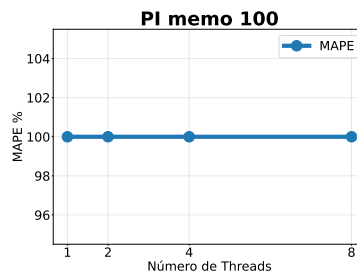
(c) Desempenho de PI com `perfo_fini` e taxa de descarte de 1000.



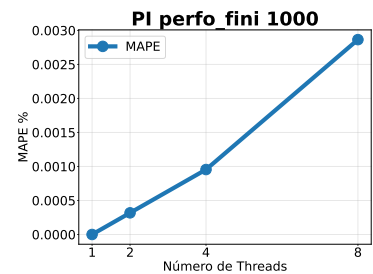
(d) Erro Médio de PI com `perfo_large` e taxa de descarte de 50.



(e) Erro Médio de PI com `perfo_init` e limiar de 100.



(f) Erro Médio de PI com `perfo_fini` e taxa de descarte de 1000.



Fonte: Autoria própria (2026).

2,69 segundos gastos pela execução sem aproximação com uma única *thread*. Esse valor indica um ganho de desempenho na ordem de $80\times$. A análise dos dados de perfilamento confirmou que essa aceleração ocorre devido à drástica redução na quantidade de ciclos de processamento executados. Em relação à qualidade dos resultados, observou-se uma baixa perda de acurácia mesmo com o aumento da taxa de descarte. Os valores obtidos indicam que o algoritmo consegue tolerar a remoção de parte das iterações sem comprometer significativamente a saída produzida. Esse comportamento está diretamente relacionado à natureza do algoritmo, que por trabalhar de maneira estatística apresenta resiliência à eliminação de iterações.

Já a cláusula `memo` apresentou o pior cenário geral, com degradação de desempenho e resultados com um erro de 100% em quase todos os testes. Em termos de tempo, a configuração com 1 *thread* demorou 23,73 segundos para concluir, um valor quase $10\times$ mais lento que a execução sem aproximação da aplicação. Essa lentidão é justificada pela natureza do método de Monte Carlo. O cálculo exige a geração ininterrupta de valores aleatórios independentes, o que impede a existência de repetições previsíveis de entrada e saída. A tentativa de aplicar memoização forçou o sistema a realizar buscas constantes e registrar novos valores em uma tabela de controle, gerando uma grande sobrecarga de processamento. Além de não acelerar a aplicação, a memoização de números que deveriam ser aleatórios quebra o princípio estatístico do algoritmo, o que justifica a falha completa na precisão matemática do programa.

5.2 Avaliação das Técnicas Empregadas

A análise conjunta das aplicações revela padrões de comportamento consistentes para cada técnica de aproximação inserida no ambiente paralelo. De maneira geral, a diretiva `perfo_large` se destacou ao gerar ganhos de desempenho em quase todas as aplicações avaliadas. Essa técnica reduz drasticamente o volume total de instruções processadas ao alterar a variável de indução do laço principal. Essa diminuição de carga computacional proporcionou diminuição na quantidade de ciclos de execução e, consequentemente, acelerações no tempo de execução total, superando com facilidade o custo de sincronização das *threads*. Contudo, esse salto de velocidade cobra um preço alto na qualidade das saídas. Na maioria dos testes, a abordagem causou uma degradação severa na acurácia do programa, chegando a valores muito próximo de 100% de erro.

A técnica `memo` demonstrou possuir um bom potencial de aceleração, mas ficou evidente que ela não pode ser aplicada em qualquer situação. O sucesso da memoização depende da previsibilidade das entradas e do custo inerente à região aproximada. Em laços com poucas instruções ou em algoritmos que exigem dados estritamente aleatórios, como o método de Monte Carlo, o *overhead* introduzido pelo gerenciamento interno das estruturas de dados supera qualquer benefício de reutilização de valores. Nesses cenários, a cláusula gera lentidão e pode até mesmo quebrar o algoritmo da aplicação. Sua adoção exige um cálculo cuidadoso para garantir que o tempo economizado ao evitar recomputações seja superior ao tempo gasto com buscas em memória.

As abordagens `perfo_init` e `perfo_fini` também trouxeram resultados interessantes, comportando-se de forma quase idêntica entre si. Diferente da técnica `perfo_large`, essas estratégias conseguem manter níveis muito altos de qualidade nas saídas quando submetidas a configurações moderadas, pois preservam a maioria do espaço de iteração. No entanto, elas dependem obrigatoriamente de uma alta taxa de descarte para apresentar ganhos reais de desempenho. Em taxas baixas, a economia de tempo gerada ao saltar as iterações iniciais ou finais não justifica o uso da aproximação na região. Somente quando há um grande salto no processamento é que o ganho de tempo se torna expressivo, o que acaba sacrificando de forma proporcional a precisão do programa.

Por fim, a cláusula `fastmath` foi uma estratégia que não introduziu ganhos significativos de desempenho em nenhum dos cenários de teste. Embora tenha preservado a integridade dos cálculos e a similaridade visual dos dados com uma margem de erro mínima, as otimizações de ponto flutuante falharam em acelerar as aplicações práticas. Em alguns programas, a necessidade de isolar a região do código para aplicar essas otimizações causou um leve aumento no tempo total da execução.

6 CONSIDERAÇÕES FINAIS

A CA consolida-se como um método essencial para suprir a crescente demanda por desempenho computacional e eficiência energética, especialmente diante das limitações físicas na evolução do *hardware*. Técnicas de aproximação foram propostas para suprir essa demanda, como é o caso de: perfuração de laços, a memoização aproximada e o relaxamento em operações de ponto flutuante. Neste trabalho estas técnicas foram introduzidas como uma extensão para a API `OpenMP` na infraestrutura do `LLVM`, o objetivo foi facilitar a integração e o controle desses algoritmos aproximados em aplicações paralelas.

A validação ocorreu por meio de experimentos computacionais envolvendo sete aplicações distintas, nas quais foram analisados o tempo total de execução e as métricas de qualidade dos resultados gerados. De maneira geral, a cláusula `perfo_large`, que aplica a perfuração de laços modificando diretamente a variável de indução, obteve os ganhos de desempenho mais expressivos, reduzindo significativamente o tempo de execução na maioria dos cenários, ainda que ao custo de uma degradação na acurácia das saídas.

Entre as demais abordagens, as estratégias `perfo_init` e `perfo_fini` atuaram como alternativas intermediárias, entregando ganhos de desempenho moderados, porém inferiores aos alcançados pela `perfo_large`. A cláusula `memo` demonstrou um bom potencial de aceleração, mas restrito a cenários específicos com alta previsibilidade de entrada. Por fim, a diretiva `fastmath` não superou o desempenho da execução exata padrão.

Como trabalhos futuros, sugere-se estender o suporte dessas cláusulas de aproximação para o nível de GPU, o que permitiria explorar o paralelismo massivo desses aceleradores em conjunto com os benefícios de desempenho da computação aproximada.

REFERÊNCIAS

- ADHIKARI, A. N. *et al.* Modeling large regions in proteins: Applications to loops, termini, and folding. *In: PROTEIN SCIENCE*. 21 n. 1., 2012. **Anais [...]** Wiley Online Library, 2012. p. 107–121. ISSN 0961-8368, 1469-896X. Disponível em: <https://onlinelibrary.wiley.com/doi/10.1002/pro.767>.
- ALRAWAIS, A. Parallel programming models and paradigms: OpenMP analysis. *In: 2021 5TH INTERNATIONAL CONFERENCE ON COMPUTING METHODOLOGIES AND COMMUNICATION (ICCMC)*. 2021. **Anais [...]** IEEE, 2021. p. 1022–1029. Disponível em: <http://dx.doi.org/10.1109/ICCMC51019.2021.9418401>.
- AXLER, S. **Linear Algebra Done Right**. Springer International Publishing, 2015. (Undergraduate Texts in Mathematics). ISBN 978-3-319-11079-0 978-3-319-11080-6. Disponível em: <https://link.springer.com/10.1007/978-3-319-11080-6>.
- CHE, S. *et al.* Rodinia: A benchmark suite for heterogeneous computing. *In: 2009 IEEE INTERNATIONAL SYMPOSIUM ON WORKLOAD CHARACTERIZATION (IISWC)*. 2009. **Anais [...]** [S.l.: s.n.], 2009. p. 44–54.
- CHIPPA, V. K. *et al.* Analysis and characterization of inherent application resilience for approximate computing. *In: PROCEEDINGS OF THE 50TH Annual Design Automation Conference*. 2013. **Anais [...]** ACM, 2013. p. 1–9. ISBN 978-1-4503-2071-9. Disponível em: <https://dl.acm.org/doi/10.1145/2463209.2488873>.
- DALLOO, A. M. *et al.* Approximate computing: Concepts, architectures, challenges, applications, and future directions. **IEEE Access**, Institute of Electrical and Electronics Engineers (IEEE) v. 12,, p. 146022–146088, 2024.
- Debian Project. **Benchmarks Game — benchmarksgame-team.pages.debian.net**. [S.l.]: , 2018. <https://benchmarksgame-team.pages.debian.net/benchmarksgame/performance/mandelbrot.html>. Acesso em: 20 out. 2025.
- DERICHE, R. Using Canny's criteria to derive a recursively implemented optimal edge detector, . v. 1, n. 2, p. 167–187, 1987. ISSN 0920-5691, 1573-1405. Disponível em: <http://link.springer.com/10.1007/BF00123164>.
- DEVANEY, R. L. The mandelbrot set, the farey tree, and the fibonacci sequence. **The American Mathematical Monthly**, Informa UK Limited v. 106, n. 4, p. 289–302, abr. 1999. ISSN 1930-0972. Disponível em: <http://dx.doi.org/10.1080/00029890.1999.12005046>.
- FABRÍCIO, J. **Software and Hardware Interfaces for Approximate Memories**. 2022. Tese de Doutorado em Ciência da Computação 2022. Disponível em: <https://repositorio.unicamp.br/Busca/Download?codigoArquivo=548558>.
- FABRÍCIO, J. *et al.* AxRAM: A lightweight implicit interface for approximate data access, . v. 113,, p. 556–570, 2020. ISSN 0167739X. Disponível em: <https://linkinghub.elsevier.com/retrieve/pii/S0167739X20301060>.
- FELZMANN, I. *et al.* Special Session: How much quality is enough quality? A case for acceptability in approximate designs. *In: 2021 IEEE 39TH International Conference ON Computer Design (ICCD)*. 2021. **Anais [...]** IEEE, 2021. p. 5–8. ISBN 978-1-6654-3219-1. Disponível em: <https://ieeexplore.ieee.org/document/9643638/>.

FINK, Z. *et al.* HPAC-Offload: Accelerating HPC Applications with Portable Approximate Computing on the GPU. *In: PROCEEDINGS OF THE International Conference FOR High Performance Computing, Networking, Storage AND Analysis*. 2023. **Anais [...]** ACM, 2023. p. 1–14. ISBN 979-8-4007-0109-2. Disponível em: <https://dl.acm.org/doi/10.1145/3581784.3607095>.

FLYNN, M. J. Some Computer Organizations and Their Effectiveness, . C-21, n. 9, p. 948–960, 1972. ISSN 0018-9340. Disponível em: <http://ieeexplore.ieee.org/document/5009071/>.

GENTRYX. **2D von Neumann Stencil.svg - Wikimedia Commons**. [S.l.]: , 2011. https://commons.wikimedia.org/wiki/File:2D_von_Neumann_Stencil.svg. Acesso em: 20 out. 2025.

GNU Project. **Optimize Options (Using the GNU Compiler Collection (GCC) 3.4.3)**. [S.l.]: , 2004. Online. Disponível em: <https://gcc.gnu.org/onlinedocs/gcc-3.4.3/gcc/Optimize-Options.html>. Acesso em: 17 nov. 2025.

GONÇALVES, R. *et al.* OpenMP is not as easy as it appears. *In: .* 2016, Koloa, Hawaii. **Anais [...]** Koloa, Hawaii: IEEE, 2016.

GRIGORIAN, B.; REINMAN, G. Accelerating Divergent Applications on SIMD Architectures Using Neural Networks, . v. 12, n. 1, p. 1–23, 2015. ISSN 1544-3566, 1544-3973. Disponível em: <https://dl.acm.org/doi/10.1145/2717311>.

HENNESSY, J.; PATTERSON, D. **Computer Architecture: A Quantitative Approach**. Elsevier Science, 2017. (The Morgan Kaufmann Series in Computer Architecture and Design). ISBN 978-0-12-811905-1. Disponível em: <https://books.google.com.br/books?id=MBQFuAEACAAJ>.

HENNESSY, J. L.; PATTERSON, D. A. A new golden age for computer architecture. **Communications of the ACM**, Association for Computing Machinery (ACM) v. 62, n. 2, p. 48–60, jan. 2019. ISSN 1557-7317. Disponível em: <http://dx.doi.org/10.1145/3282307>.

HESSE, H. **Steppenwolf**. [S.l.]: Penguin Classics, 2025. (Penguin Classics). ISBN 9780143137825.

INCHEOL. **Kmeans - Wikimedia Commons**. [S.l.]: , 2015. https://commons.wikimedia.org/wiki/File:Kmeans_wronginit2.PNG. Acesso em: 20 out. 2025.

Intel. **Alphabetical Option List — intel.com**. [S.l.]: , 2026. Online. Disponível em: <https://www.intel.com/content/www/us/en/docs/dpcpp-cpp-compiler/developer-guide-reference/2023-0/alphabetical-option-list.html>. Acesso em: 08 mar. 2026.

KMHKMH. **Pi Monte Carlo - Wikimedia Commons**. [S.l.]: , 2023. https://commons.wikimedia.org/wiki/File:Pi_monte_carlo_all.gif. Acesso em: 20 out. 2025.

LASHGAR, A.; ATOOFIAN, E.; BANIASADI, A. Loop Perforation in OpenACC. *In: 2018 IEEE Intl Conf ON Parallel & Distributed Processing WITH Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*. 2018. **Anais [...]** IEEE, 2018. p. 163–170. ISBN 978-1-7281-1141-4. Disponível em: <https://ieeexplore.ieee.org/document/8672231/>.

LEON, V. *et al.* Approximate Computing Survey, Part II: Application-Specific & Architectural Approximation Techniques and Applications, . Association for Computing Machinery (ACM) v. 57, n. 7, p. 1–36, 2025. ISSN 0360-0300, 1557-7341. Disponível em: <https://dl.acm.org/doi/10.1145/3711683>.

LEON, V. *et al.* Approximate Computing Survey, Part I: Terminology and Software & Hardware Approximation Techniques, . v. 57, n. 7, p. 1–36, 2025. ISSN 0360-0300, 1557-7341. Disponível em: <https://dl.acm.org/doi/10.1145/3716845>.

LI, S.; PARK, S.; MAHLKE, S. Sculptor: Flexible Approximation with Selective Dynamic Loop Perforation. *In: PROCEEDINGS OF THE 2018 International Conference ON Supercomputing*. 2018. **Anais [...]** ACM, 2018. p. 341–351. ISBN 978-1-4503-5783-8. Disponível em: <https://dl.acm.org/doi/10.1145/3205289.3205317>.

LLVM. **Documentation OpenMP in LLVM**. [S.l.]: , 2026. Disponível em: <https://openmp.llvm.org/>. Acesso em: 08 mar. 2026.

MACQUEEN, J. Some methods for classification and analysis of multivariate observations. *In: PROCEEDINGS OF THE FIFTH BERKELEY SYMPOSIUM ON MATHEMATICAL STATISTICS AND PROBABILITY, VOLUME 1: STATISTICS*. 1967, Berkeley, CA. **Anais [...]** Berkeley, CA: University of California Press, 1967. p. 281–297.

MATTSON, T. G.; HE, Y. H.; KONIGES, A. E. **The OpenMP Common Core**. [S.l.]: , 2019.

MICHIE, D. “memo” functions and machine learning. **Nature**, Nature Publishing Group UK London v. 218, n. 5136, p. 19–22, 1968.

MICROSOFT. **float_control pragma**. [S.l.]: , 2021. Online. Disponível em: <https://learn.microsoft.com/en-us/cpp/preprocessor/float-control?view=msvc-170>. Acesso em: 17 nov. 2025.

MITTAL, S. A Survey of Techniques for Approximate Computing, . v. 48, n. 4, p. 1–33, 2016. ISSN 0360-0300, 1557-7341. Disponível em: <https://dl.acm.org/doi/10.1145/2893356>.

MONNIAUX, D. The pitfalls of verifying floating-point computations. **ACM Transactions on Programming Languages and Systems**, v. 30, n. 3, p. 1–41, maio 2008. ISSN 0164-0925, 1558-4593.

MORETTIN, P. A.; BUSSAB, W. d. O. **Estatística Básica**. 6. ed. rev. atual. ed. [S.l.]: Saraiva, 2010. ISBN 978-85-02-08177-2.

OLIVEIRA, J. B.; GONÇALVES, R. A.; FILHO, J. F. Multithread Approximation: A new OpenMP construct. *In: ANAIS DO XXV Simpósio EM Sistemas Computacionais DE Alto Desempenho (SSCAD 2024)*. 2024. **Anais [...]** Sociedade Brasileira de Computação, 2024. p. 372–383. Disponível em: <https://sol.sbc.org.br/index.php/sscad/article/view/31012>.

OLIVEIRA, J. B.; GONÇALVES, R. A.; FILHO, J. F. OpenMP em direção à aproximação: Loop perforation e multithreading. *In: ANAIS DA XV ESCOLA REGIONAL DE ALTO DESEMPENHO DE SÃO PAULO (ERAD-SP 2024)*. 2024. **Anais [...]** Sociedade Brasileira de Computação, 2024. p. 49–52. Disponível em: <https://sol.sbc.org.br/index.php/eradsp/article/view/28732>.

OLIVEIRA, J. B. de; GONÇALVES, R. A.; FILHO, J. F. Multithread approximation: An OpenMP constructor. **Concurrency and Computation: Practice and Experience**, Wiley v. 38, n. 4, p. e70584, fev. 2026. Disponível em: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cpe.70584>.

OLIVEIRA, J. V. B. D.; GONÇALVES, R. A.; FILHO, J. F. Anotação de código de alto nível para aproximação de sincronização e dados. *In: ANAIS DO XIV SEMINÁRIO DE EXTENSÃO E INOVAÇÃO & XXIX SEMINÁRIO DE INICIAÇÃO CIENTÍFICA E TECNOLÓGICA DA UTFPR – SEI/SICITE*. 2024, Francisco Beltrão, PR. **Anais [...]** Francisco Beltrão, PR: UTFPR, 2024. Acesso em: 17/11/2025. Disponível em: <https://www.even3.com.br/anais/seisicite2024/965072-ANOTACAO-DE-CODIGO-DE-ALTO-NIVEL-PARA-APROXIMACAO-DE-SINCRONIZACAO-E-DADOS>.

OPENMP. **OpenMP API Specification**. [S.l.]: , 2018. Disponível em: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5.0.pdf>. Acesso em: 17 nov. 2025.

OpenMP Architecture Review Board. **OpenMP Application Programming Interface**. [S.l.], 2021. Disponível em: <http://www.openmp.org/>. Acesso em: 08 mar. 2026.

ORGANIZATION, O. **The OpenACC Application Programming Interface**. [S.l.]: , 2020. Version 3.0. Disponível em: <https://www.openacc.org/specification>. Acesso em: 08 mar. 2026.

PADUA, D. Posix threads (pthreads). *In: .* [S.l.: s.n.], 2011. p. 1592–1593. ISBN 9780387097657.

PARASYRIS, K. *et al.* HPAC: Evaluating approximate computing techniques on HPC OpenMP applications. *In: PROCEEDINGS OF THE International Conference FOR High Performance Computing, Networking, Storage AND Analysis*. 2021. **Anais [...]** ACM, 2021. p. 1–14. ISBN 978-1-4503-8442-1. Disponível em: <https://dl.acm.org/doi/10.1145/3458817.3476216>.

POUCHET, L.-N. **PolyBench**. [S.l.]: , 2011. <https://sourceforge.net/projects/polybench/>. Acesso em: 13 out. 2025.

REIS, L.; WANNER, L. Functional approximation and approximate parallelization with the accept compiler. *In: .* 2021. **Anais [...]** [S.l.: s.n.], 2021. p. 188–197.

REIS, L.; WANNER, L.; RIGO, S. Towards Just-In-Time Software Approximations. *In: ANAIS DO XXV Simpósio EM Sistemas Computacionais DE Alto Desempenho (SSCAD 2024)*. 2024. **Anais [...]** Sociedade Brasileira de Computação, 2024. p. 360–371. Disponível em: <https://sol.sbc.org.br/index.php/sscad/article/view/31011>.

SAMPSON, A. *et al.* ACCEPT: A programmer-guided compiler framework for practical approximate computing. **University of Washington Technical Report UW-CSE-15-01**, v. 1, n. 2, p. 1–14, 2015.

SIDIROGLOU-DOUSKOS, S. *et al.* Managing performance vs. accuracy trade-offs with loop perforation. *In: PROCEEDINGS OF THE 19TH ACM SIGSOFT SYMPOSIUM AND THE 13TH European CONFERENCE ON Foundations OF SOFTWARE ENGINEERING*. 2011. **Anais [...]** ACM, 2011. p. 124–134. ISBN 978-1-4503-0443-6. Disponível em: <https://dl.acm.org/doi/10.1145/2025113.2025133>.

STATISTICS, L. **Pearson Correlation Coefficient and associated scatterplots.png - Wikimedia Commons**. [S.l.]: , 2019. https://commons.wikimedia.org/wiki/File:Pearson_Correlation_Coefficient_and_associated_scatterplots.png. Acesso em: 20 out. 2025.

SZÉP, G. **Handbook of Mathematics**. 5th ed. ed. [S.l.]: Springer, 2007. ISBN 978-3-540-72121-5.

TANENBAUM, A. S. **Modern Operating Systems**. Fourth edition. [S.l.]: Pearson, 2015. ISBN 978-0-13-359162-0.

The LLVM Project. **Clang Compiler User's Manual**. [S.l.]: , 2025. Online. Disponível em: <https://clang.llvm.org/docs/UsersManual.html#cmdoption-ffast-math>. Acesso em: 17 nov. 2025.

TZIANZIOULIS, G.; HARDAVELLAS, N.; CAMPANONI, S. Temporal Approximate Function Memoization. **IEEE Micro**, v. 38, n. 4, p. 60–70, jul. 2018. ISSN 0272-1732, 1937-4143.

VASSILIADIS, V. *et al.* A programming model and runtime system for significance-aware energy-efficient computing. **ACM SIGPLAN Notices**, ACM New York, NY, USA v. 50, n. 8, p. 275–276, 2015.

VIGNA, S. Further scramblings of marsaglia's xorshift generators. **Journal of Computational and Applied Mathematics**, Elsevier v. 315, p. 175–181, 2017.

XU, Q.; MYTKOWICZ, T.; KIM, N. S. Approximate Computing: A Survey. **IEEE Design and Test**, v. 33, n. 1, p. 8–22, feb 2016. ISSN 21682356. Disponível em: <http://ieeexplore.ieee.org/document/7348659/>.